

Lecture 6

Beyond Classical Search:

Evolutionary Computation [Genetic Algorithms]

PART A

- Heuristic Search Techniques
 - Principal Heuristic Algorithms
- Genetic Algorithms (GA) OVERVIEW
 - Premise of GAs:
 - Natural Selection
 - Inheritance of Characteristics
 - Stochastic operators
 - The Metaphor

Outline

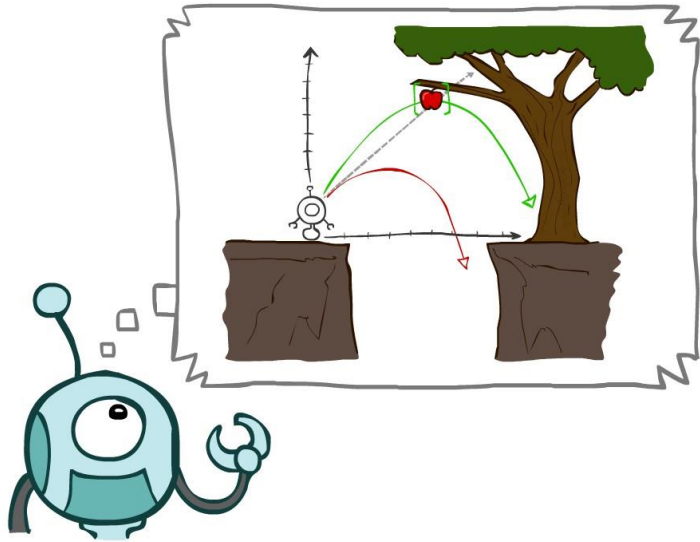


PART B

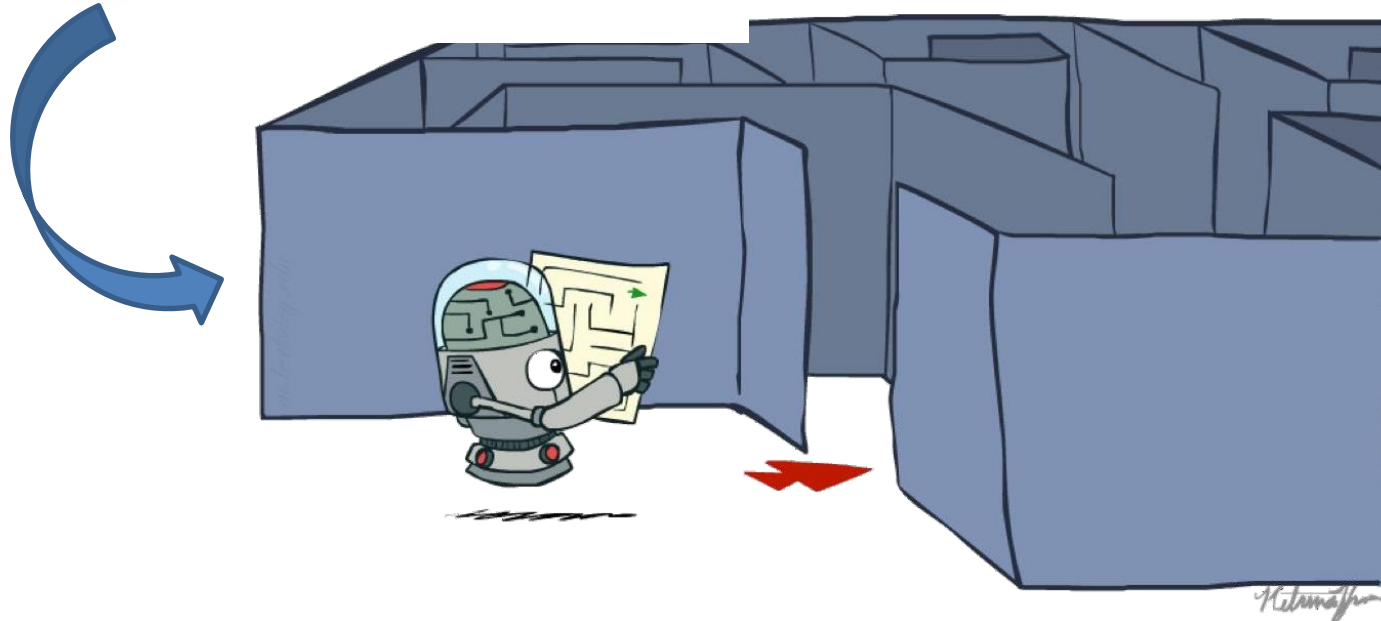
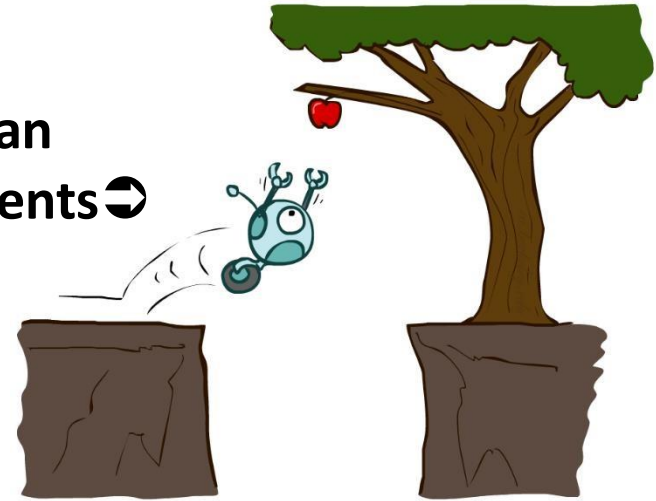
- GA Terminology
 - Chromosome
 - Encoding - Decoding
 - Simple Genetic Algorithm
 - The Evolutionary Cycle
- Example: the MAXONE problem

- Components of a GA
 - Representation (encoding)
 - Initialization
 - Selection
 - Selection by Ranking
 - Roulette Wheel Selection
 - Tournament Selection
 - Recombination (Crossover)
 - Mutation
 - Evaluation (Fitness Function)
 - Termination condition
- Another Example: The Traveling Salesman Problem (TSP)
- GA Convergence

Heuristic Search Techniques.. Recap



◀ Agents that Plan
versus Reflex Agents ▶



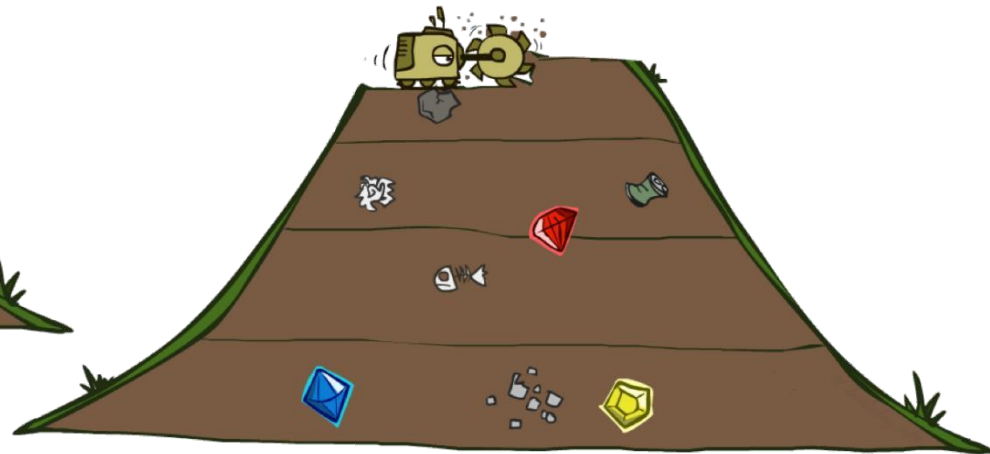
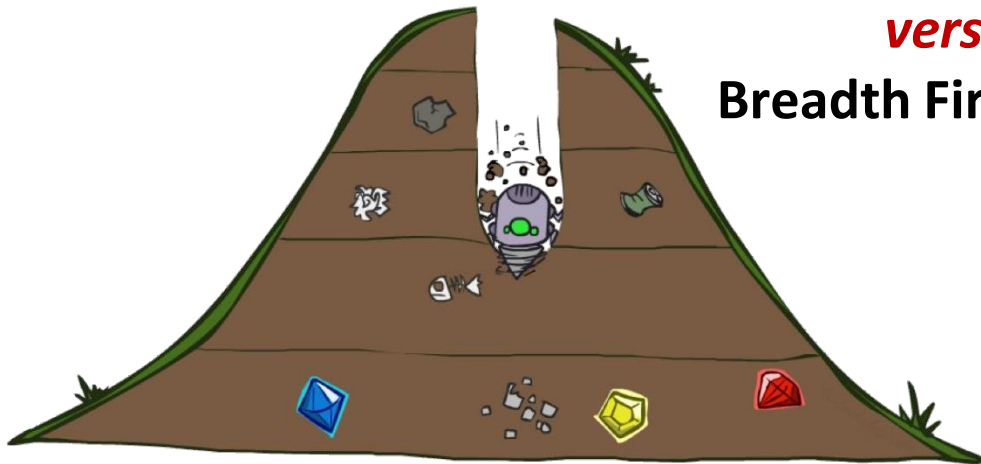
Heuristic Search Techniques

Recap: Uninformed Search

◀ Depth First Search

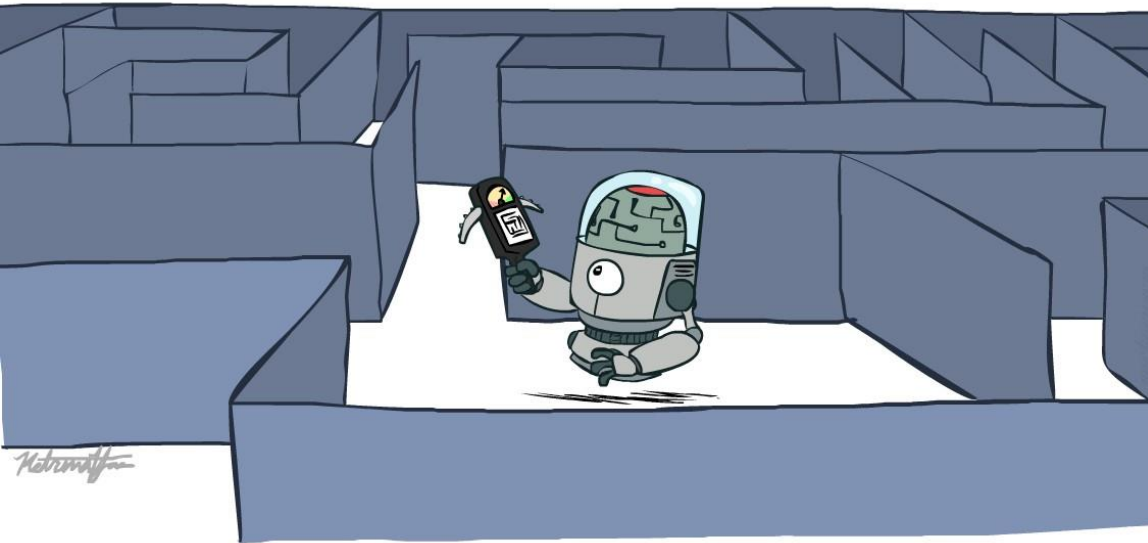
versus

Breadth First Search ▶



Heuristic Search Techniques

Recap: Informed Search

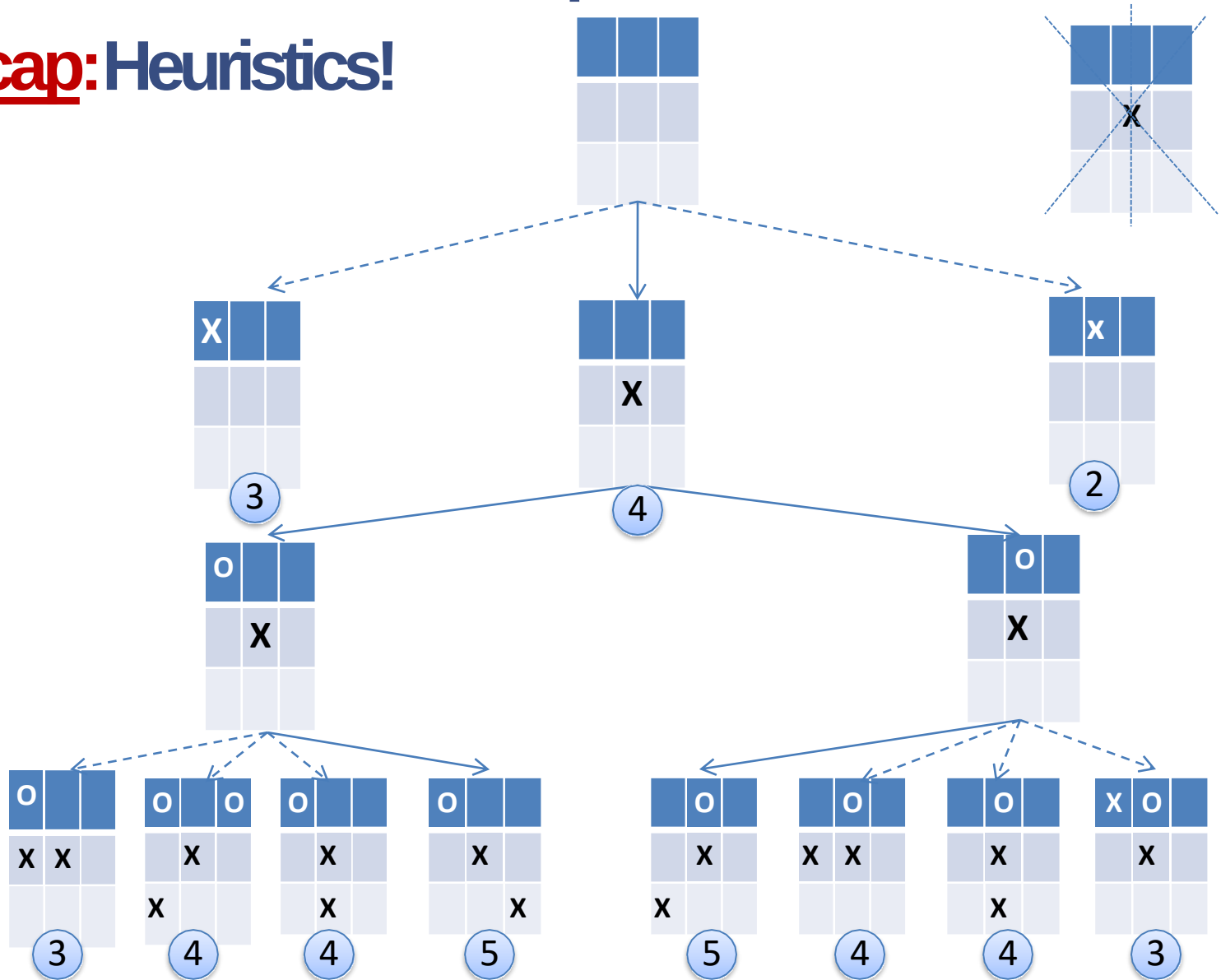


Greedy Search



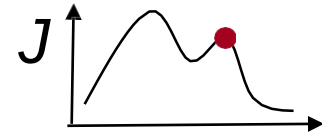
Heuristic Search Techniques

Recap: Heuristics!



Heuristic Search Techniques

Main Motivation for Heuristic Techniques:



(1) To deal with local optima and not get trapped in them.

(2) To allow optimization for systems, where the design variables are not only continuous, but discrete (categorical), integer, or even Boolean.

$x_i \notin \mathbb{R}$ $x_i = \{1, 2, 3, 4, 5\}$, $x_i = \{"A", "B", "C"\}$, $x_i = \{\text{true}, \text{false}\}$

➡ These techniques do not guarantee that global optimum can be found.

Principal Heuristic Algorithms

- Genetic Algorithms (Holland – 1975)
 - Inspired by genetics and natural selection.
- Simulated Annealing (Kirkpatrick – 1983)
 - Inspired by statistical mechanics.
- Particle Swarm Optimization (Eberhart Kennedy - 1995)
 - Inspired by the social behavior of swarms of insects or flocks of birds.

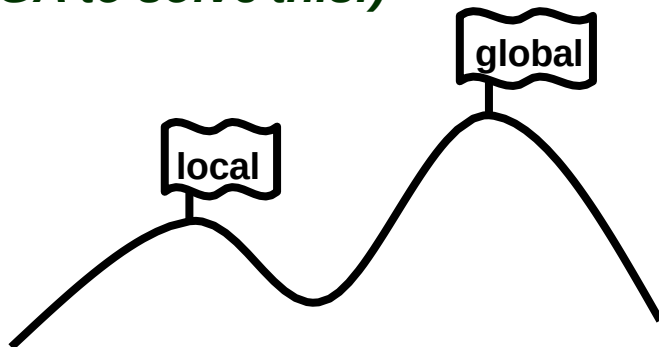


These techniques all use a combination of randomness and heuristic “rules” to guide the search for global maxima or minima.

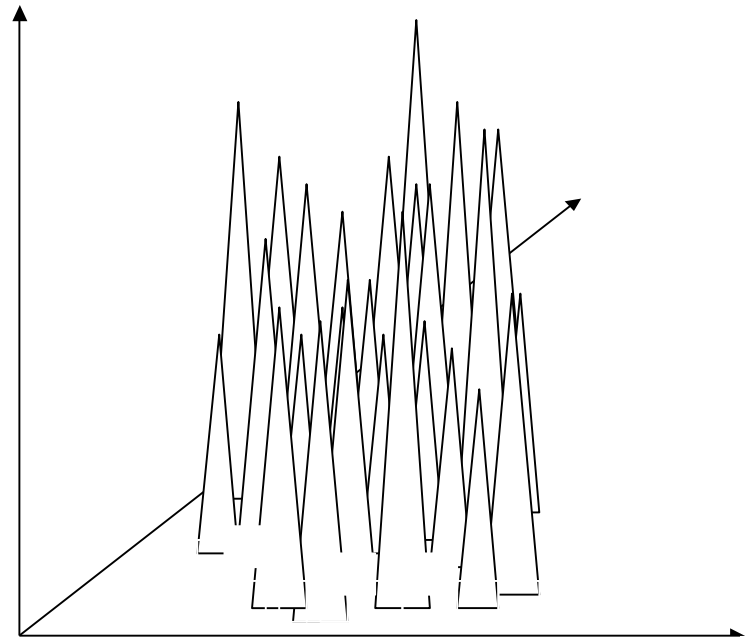
Genetic Algorithms(GA) OVERVIEW

GA's are useful for solving multidimensional problems containing many local maxima (or *minima*) in the **solution space**.

A simple optimisation problem (*no need to use a GA to solve this!*)

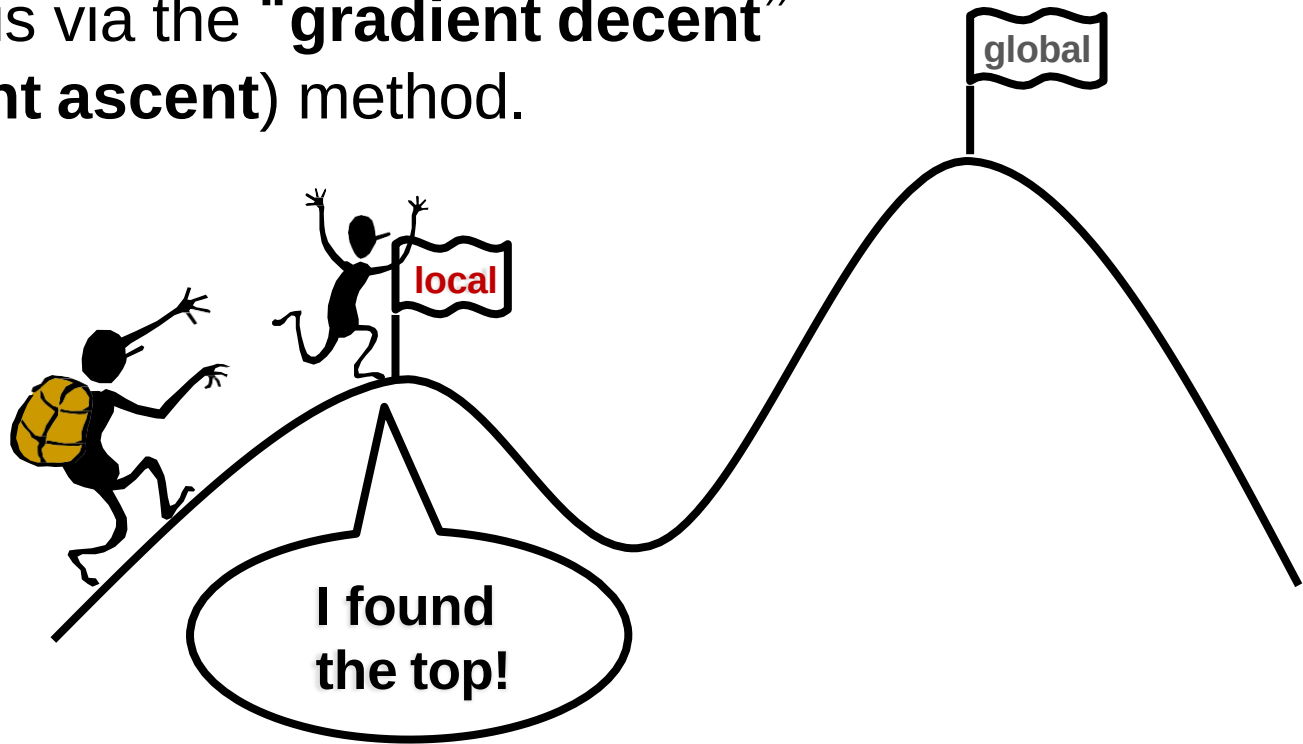


A real-world problem



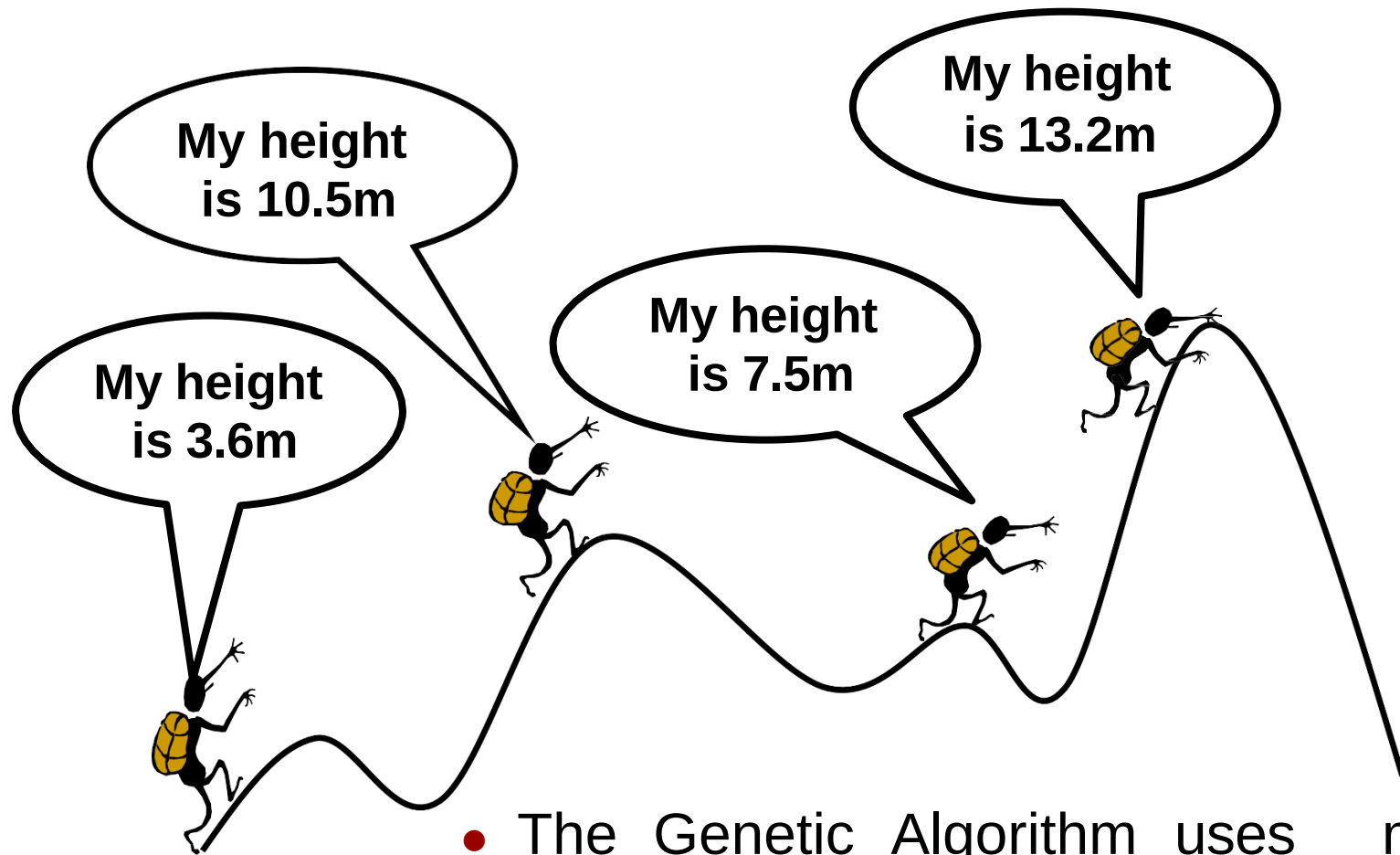
Genetic Algorithms(GA) OVERVIEW

A standard method of finding maxima or minima is via the “**gradient decent**” (**gradient ascent**) method.



- Problem: this method may find only a ..
local maxima / minima !

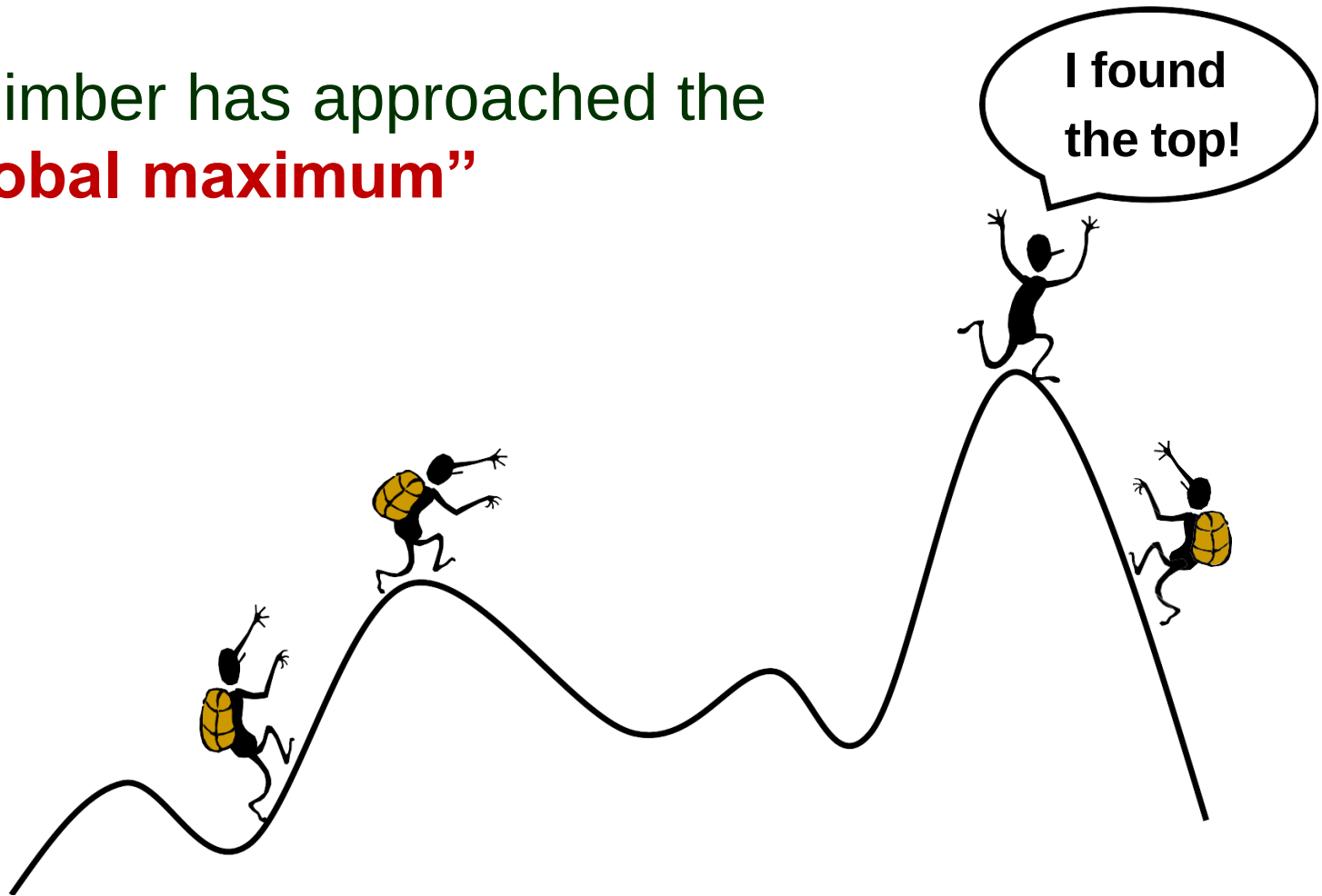
Genetic Algorithms(GA)OVERVIEW



- The Genetic Algorithm uses multiple climbers in parallel to find the global optimum (a population-based algorithm).

Genetic Algorithms(GA) OVERVIEW

A climber has approached the
“global maximum”



Genetic Algorithms(GA) OVERVIEW

- A class of probabilistic **optimization algorithms**.
- Inspired by the **biological evolution** process.
- Uses concepts of “**Natural Selection**” and “**Genetic Inheritance**” (Darwin 1859).
- Originally developed by John Holland (1975).
- Particularly **well suited for hard problems where little is known about the underlying search space.**
- *Widely-used in business, science and engineering.*

Genetic Algorithms(GA) OVERVIEW

– Evolutionary Computation(EC)

– Evolutionary Algorithms(EA)

- Evolutionary Computing (*or Evolutionary Algorithms “EA”*) is a field of science and engineering that tries to apply some phenomena that appears in the nature to the optimization.
- Most notable is the adaptation of Charles Darwin’s evolution theory in order to solve difficult search and optimization tasks.
- The method is universally applicable, since it has been successfully applied to almost all thinkable search & optimization problems in engineering, science & people’s everyday life.

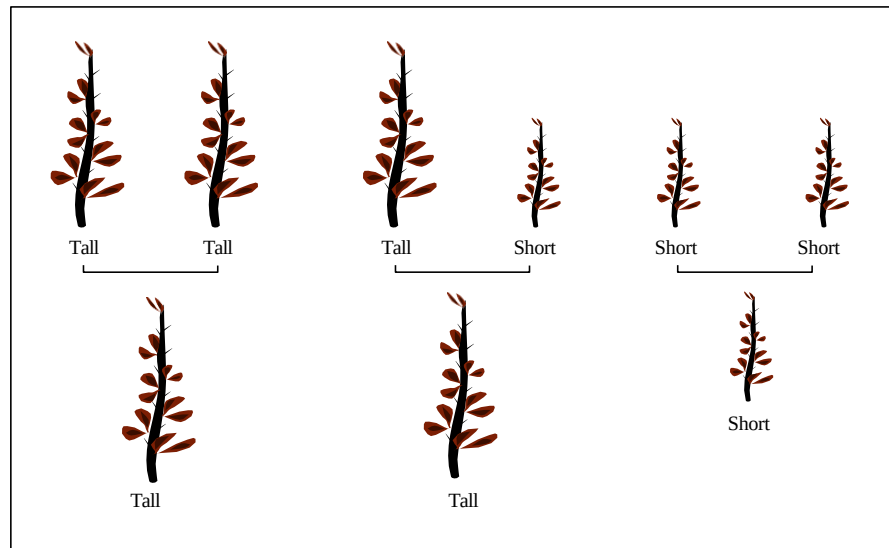
Premise of GAs: Natural Selection

- **Natural Selection** is a very successful organizing principle for optimizing individuals and populations of individuals
- If we can **mimic natural selection**, then we will be able to optimize more successfully
- A possible design of a system – as represented by its design vector $\mathbf{x}$ – can be considered as an individual who is fighting for survival within a larger population.
- Only **the fittest survive** – Fitness is assessed via objective function.

Premise of GAs: Inheritance of Characteristics

Gregor Mendel (1822-1884)

- Investigated the inheritance of characteristics (“traits”).
- Conducted extensive experiments with pea plants.
- Examined hybrids from different strains of plant.



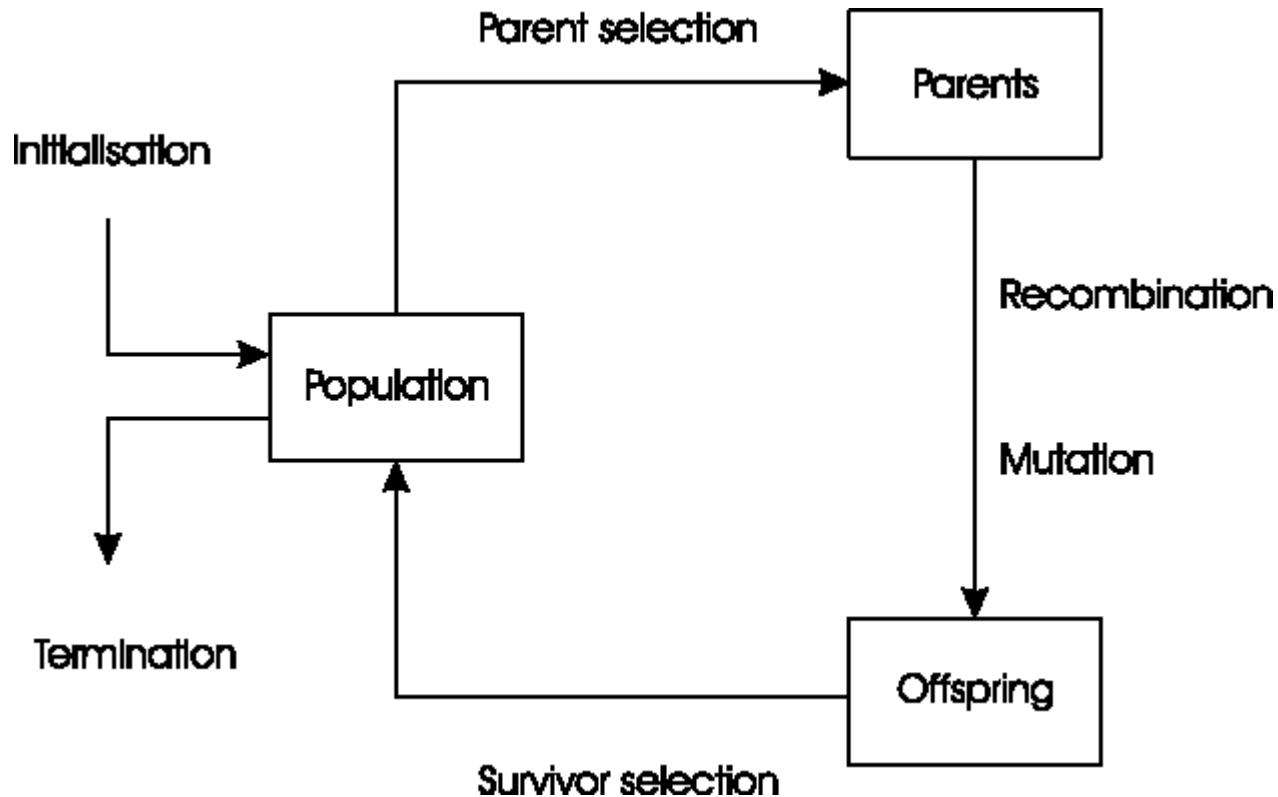
Character (gene) for tallness is dominant.
Character (gene) for shortness is recessive.

Genetic Algorithms(GA)OVERVIEW

**A genetic algorithm maintains a
population of candidate solutions for
the problem at hand,
and makes it evolve by
iteratively applying
a set of stochastic operators**

Genetic Algorithms(GA)OVERVIEW

General Scheme of Genetic Algorithms



Genetic Algorithms (GA) OVERVIEW

Stochastic Operators

Selection replicates the most successful solutions found in a population at a rate proportional to their relative **quality**.

Recombination (Crossover) decomposes two distinct solutions and then randomly mixes their parts to form novel solutions.

Mutation randomly perturbs a candidate solution.

Genetic Algorithms (GA) OVERVIEW

The Metaphor

Genetic Algorithm	Nature
Optimization Problem	Environment
Feasible Solutions	Individuals living in that environment ..
Solutions Quality (<i>fitness function</i>)	Individual's degree of adaptation to its surrounding environment

Genetic Algorithms(GA) OVERVIEW

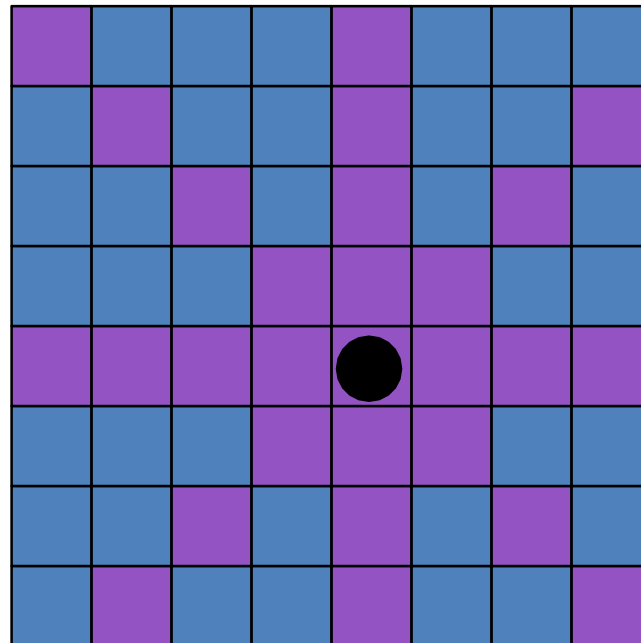
The Metaphor(cont.)

Genetic Algorithm	Nature
A set of feasible solutions	A population of organisms (<i>species</i>).
Stochastic operators	Selection, recombination, and mutation in nature's evolutionary process.
Iteratively applying a set of stochastic operators on a set of feasible solutions.	Evolution of populations to suit their environment.

A Basic Example ..

The 8 Queens Problem ..

- Place 8 queens on an 8x8 chessboard in such a way that they cannot check each other.

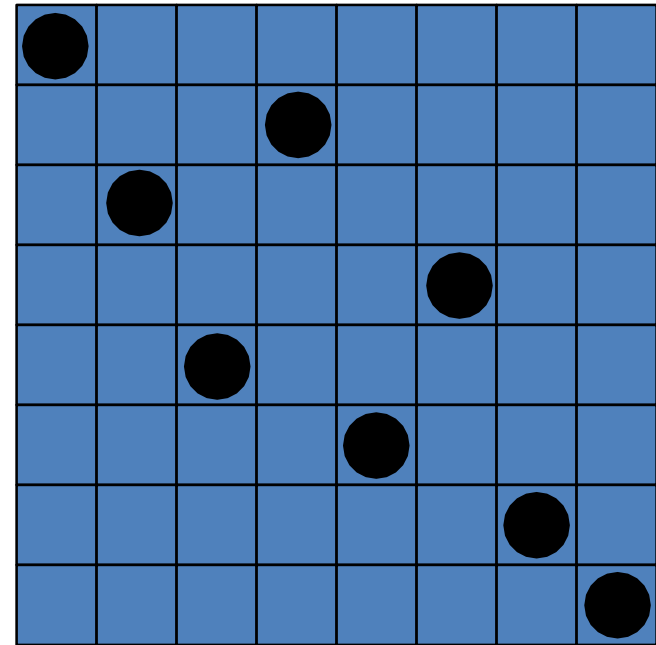


A Basic Example ..

The 8 Queens Problem: Representation ..

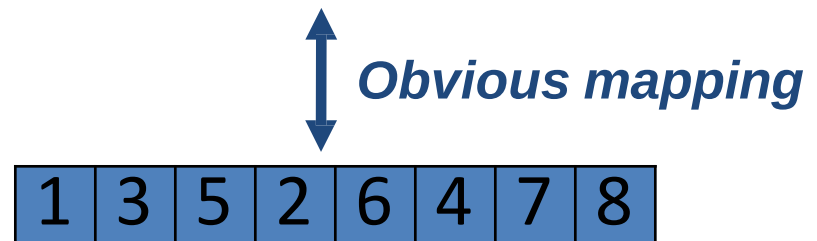
Phenotype:

A board's configuration ..



Genotype:

A permutation of
the numbers 1 : 8



A Basic Example ..

The 8 Queens Problem: *Fitness Function (Evaluation) ..*

- **Penalty of one queen:** the number of queens she can check.
- **Penalty of a configuration:** the sum of the penalties of all queens.
- **Note:** penalty is to be minimized.
- **Fitness of a configuration:** inverse penalty to be maximized.

A Basic Example ..

The 8 Queens Problem: Parents' Selection for Recombination (Crossover)..

Selecting Parents for Crossover/Recombination:

- For e.g., by picking randomly 5 parents, and then select the best two to undergo crossover.

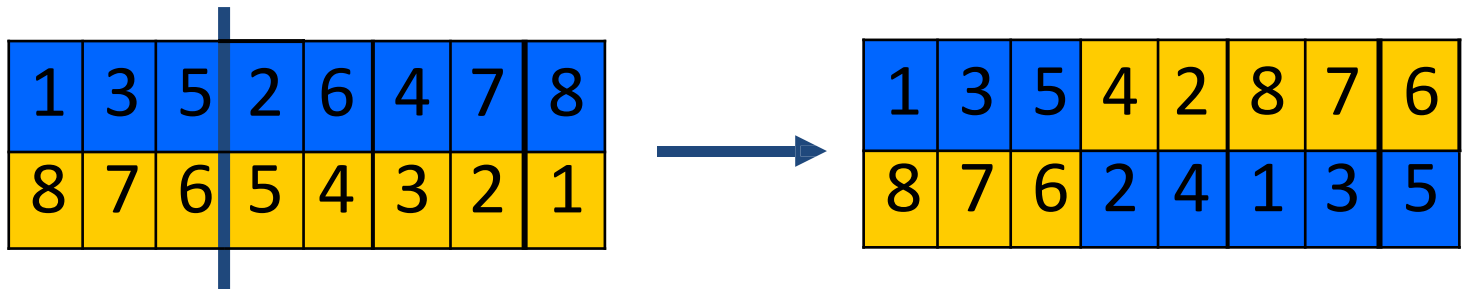
A Basic Example ..

The 8 Queens Problem:

Recombination (Crossover) ..

Combining two chromosomes (*in this case, permutations*) into two new permutations:

- Choose a random crossover point ..
- Copy first parts into both children ..
- Create the second part by inserting values from the other parent:
 - In the order they appear there (at that parent) ..
 - Beginning after the selected crossover point .. and ..
 - Skipping values already in the child.



A Basic Example ..

The 8 Queens Problem: *Mutation* ..

- Small variation in one permutation:
- *For e.g.*, swapping values of two randomly chosen positions.



A Basic Example ..

The 8 Queens Problem: Parents' Selection for Replacement ..

Survivors' Selection (*Replacement; "Creating a new generation"*)

.. For Example: when inserting a new child into the population, choose an existing member to replace by:

- Sorting the whole population by decreasing fitness.
- Enumerating this list from high to low.
- Replacing the first with a fitness lower than the given child.

A Basic Example ..

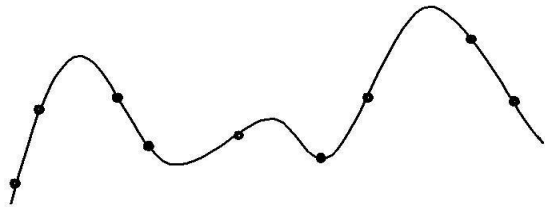
The 8 Queens Problem: *Summary ..*

Representation	Permutations
Recombination	“Cut-and-crossfill” crossover
Recombination probability	100%
Mutation	Swap
Mutation probability	80%
Parent selection	Best 2 out of random 5
Survival selection	Replace worst
Population size	100
Number of Offspring	2
Initialisation	Random
Termination condition	Solution or 10,000 fitness evaluation

Note that this is ***only one possible*** set of choices of operators and parameters.

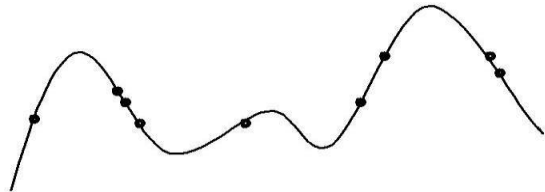
Typical Behaviour of an EA ..

Phases in optimising on a 1-dimensional fitness landscape:



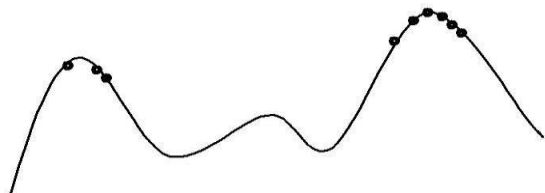
An Early Phase:

Quasi-random population distribution.



A Middle Phase:

Population arranged around/on hills.



A Late phase:

Population concentrated on high hills.

Simple Genetic Algorithm

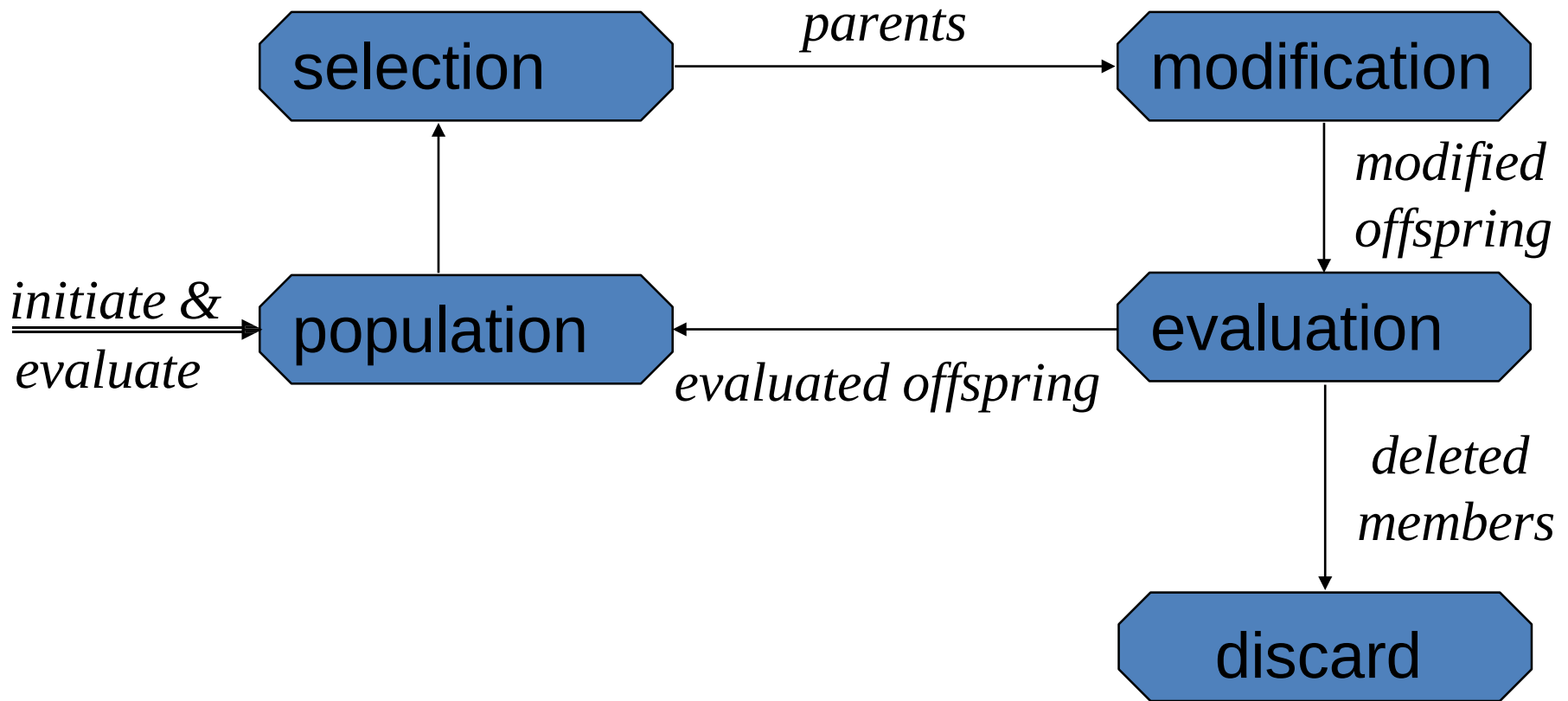
- produce an initial population of individuals
- evaluate the fitness of all individuals
- **while** termination condition not met **do**
 - select fitter individuals for reproduction
 - recombine between individuals
 - mutate individuals
 - evaluate the fitness of the modified individuals
 - generate a new population
- **End while**

Pseudocode for a Typical Genetic Algorithm (GA)

```
BEGIN
  INITIALISE population with random candidate solutions;
  EVALUATE each candidate;
  REPEAT UNTIL ( TERMINATION CONDITION is satisfied ) DO
    1 SELECT parents;
    2 RECOMBINE pairs of parents;
    3 MUTATE the resulting offspring;
    4 EVALUATE new candidates;
    5 SELECT individuals for the next generation;
  OD
END
```

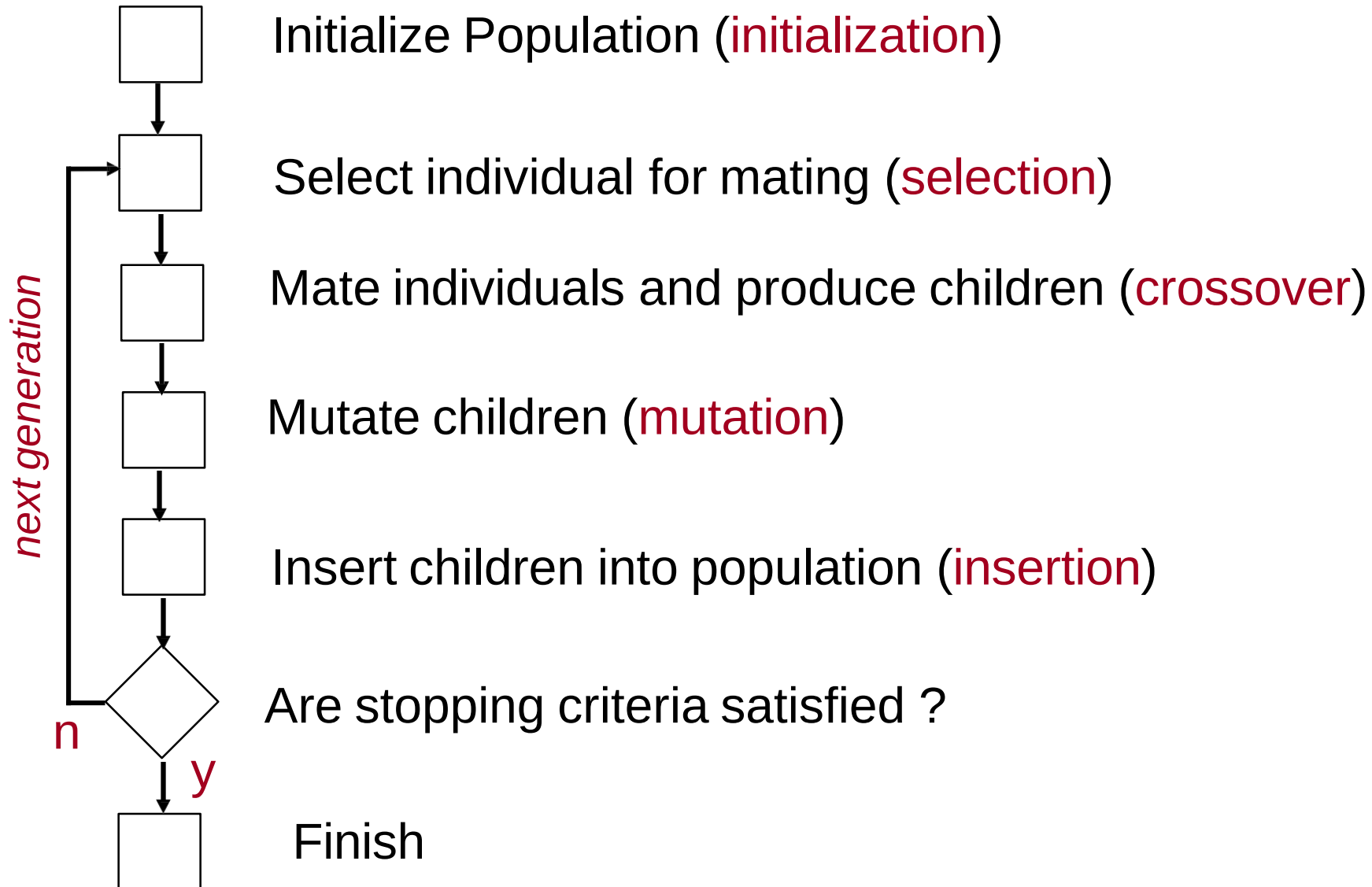

Simple Genetic Algorithm

The Evolutionary Cycle



Simple Genetic Algorithm

The Evolutionary Cycle

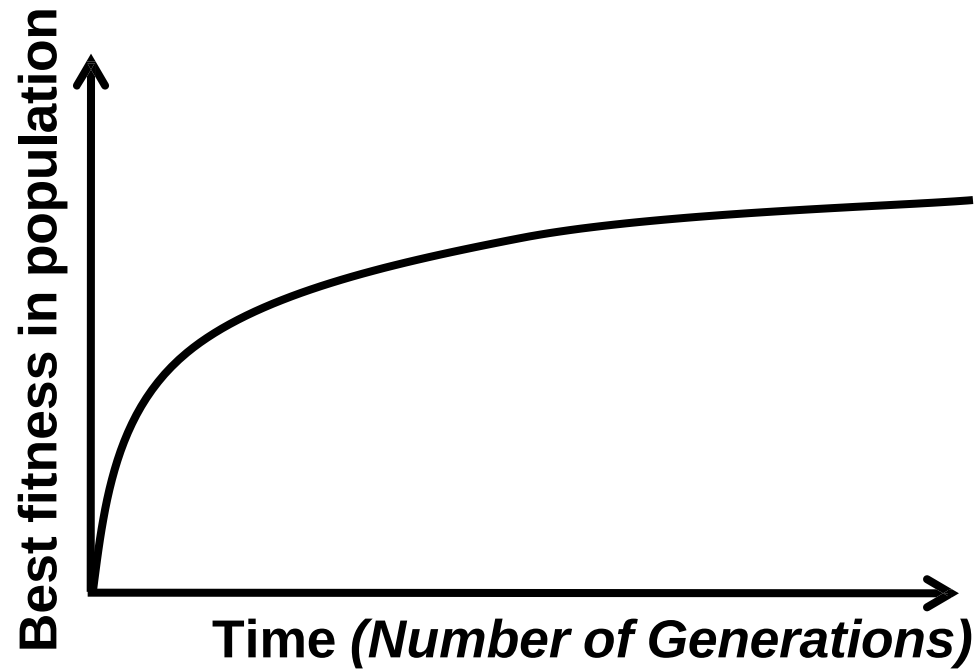


Termination Condition (Stopping Criteria)

Examples:

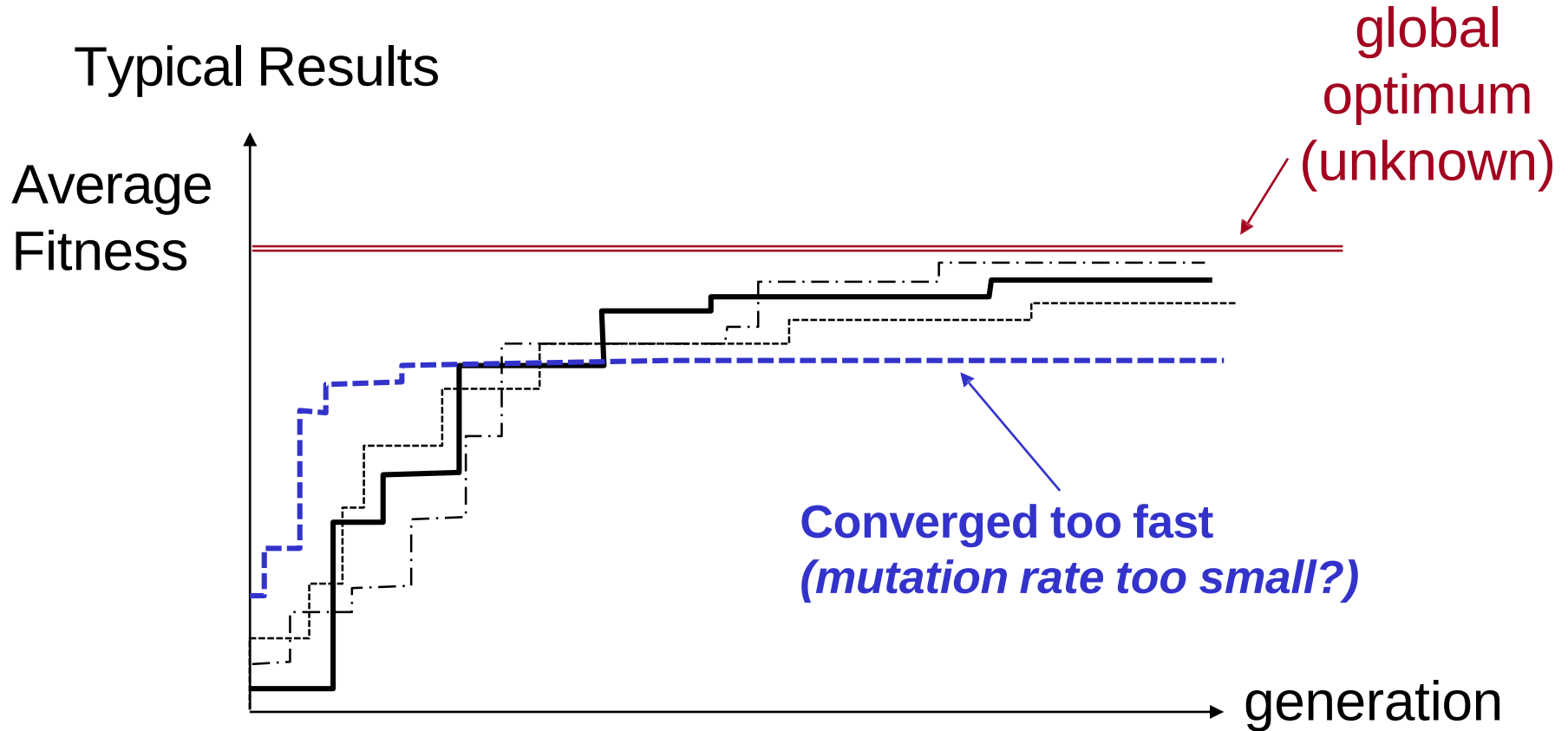
- A pre-determined number of generations or time has elapsed.
- A satisfactory solution has been achieved.
- No improvement in solution quality has taken place for a pre-determined number of generations.

Typical Run: Progression of Fitness



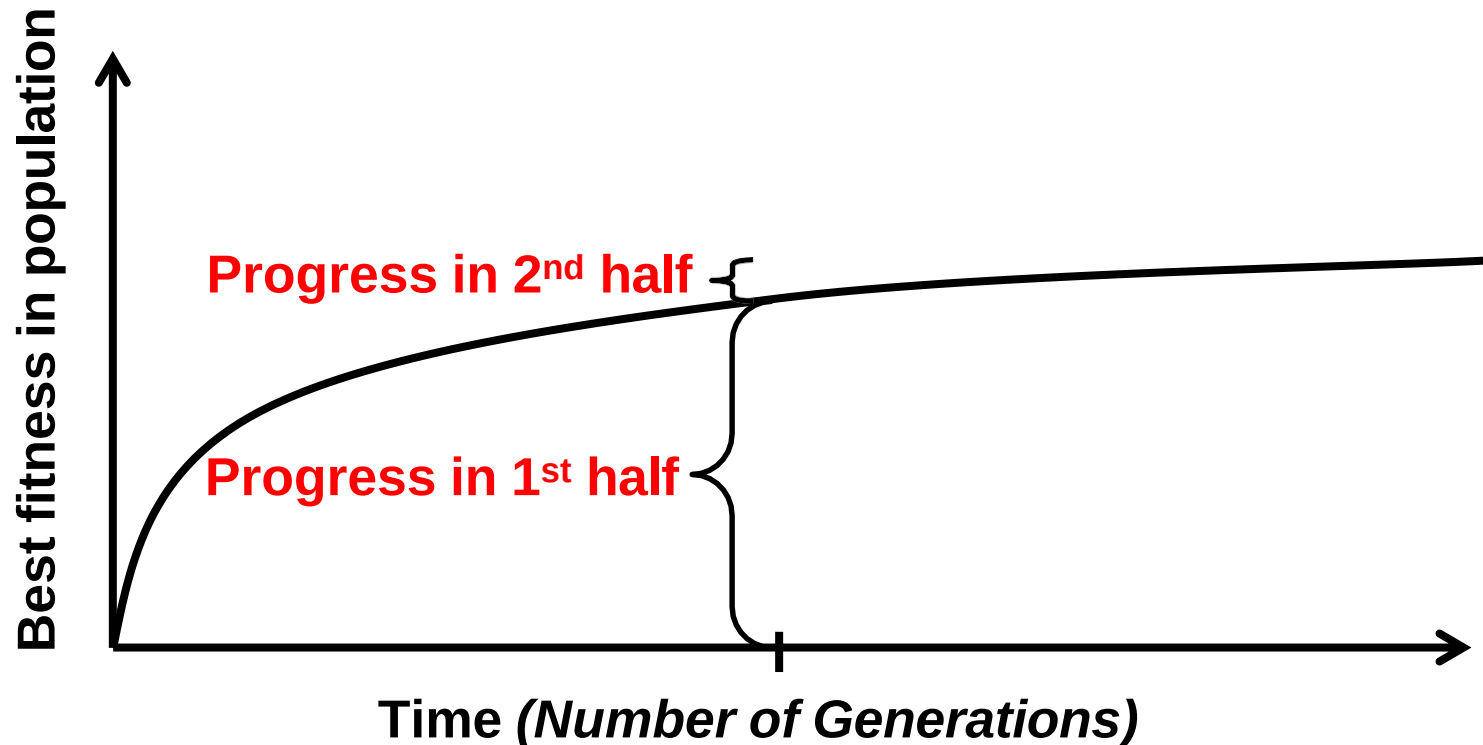
Exploration (through mutation) *versus* Exploitation (through crossover)

GA Convergence



Average performance of individuals in a population is expected to increase, as good individuals are preserved and bred and less fit individuals die out.

Are long runs beneficial?



Answer:

- it depends on how much you want the last bit of progress ..

Lecture 6 [**PART B**]

Beyond Classical Search: Evolutionary Computation [Genetic Algorithms]

Lecture is based on its counterparts in the following courses:

- *Artificial Intelligence*, **Massachusetts Institute of Technology MIT** - Prof. de Weck & Prof. Willcox Engineering Systems Division and Dept. of Aeronautics and Astronautics.
- *Artificial Intelligence*, **University of California, Berkeley**
- *Evolutionary Computing*, **University of Vaasa (Finland)**, Electrical & Energy Eng.
- *Advanced Topics in Image Analysis and Machine Learning*, **Ritsumeikan University (Kyoto – Japan)**, Faculty of Information Science and Engineering.

PART A

- Heuristic Search Techniques
 - Principal Heuristic Algorithms
- Genetic Algorithms (GA) OVERVIEW
 - Premise of GAs:
 - Natural Selection
 - Inheritance of Characteristics
 - Stochastic operators
 - The Metaphor

Outline



PART B

- GA Terminology
 - Chromosome
 - Encoding - Decoding
 - Simple Genetic Algorithm
 - The Evolutionary Cycle
- Example: the MAXONE problem

- Components of a GA
 - Representation (encoding)
 - Initialization
 - Selection
 - Selection by Ranking
 - Roulette Wheel Selection
 - Tournament Selection
 - Recombination (Crossover)
 - Mutation
 - Evaluation (Fitness Function)
 - Termination condition
- Another Example: The Traveling Salesman Problem (TSP)
- GA Convergence

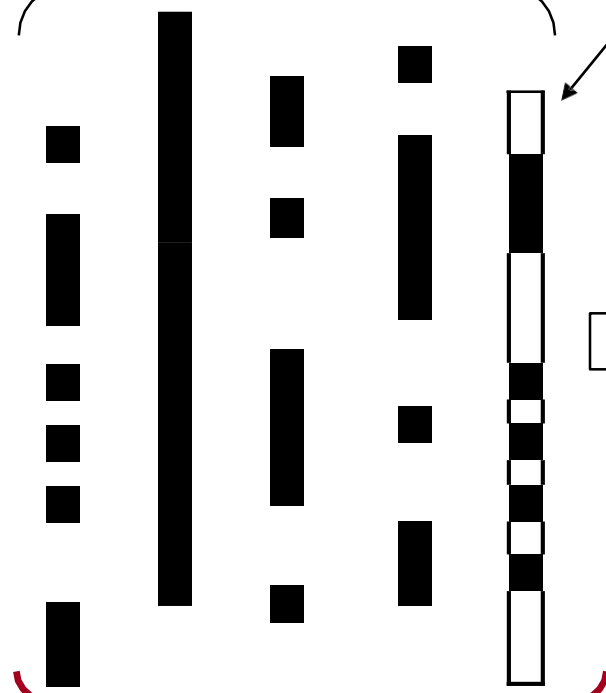
GA Terminology

Chromosome



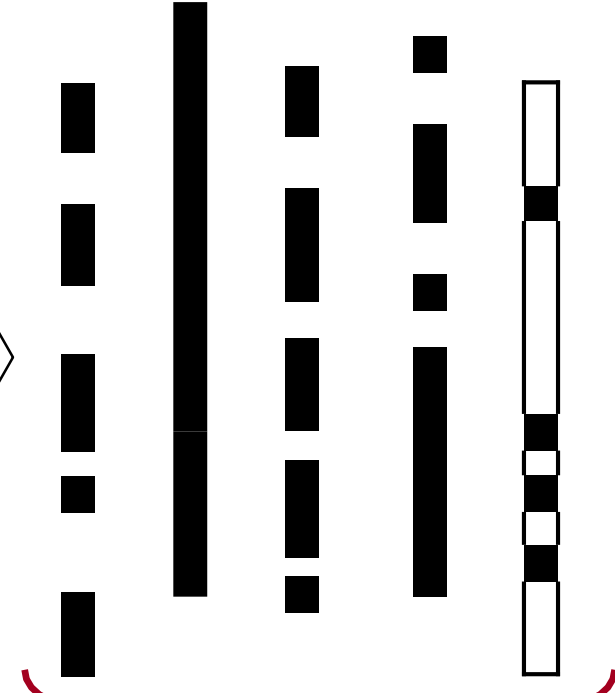
population

individual



selection
crossover
insertion
mutation

genetic
operators

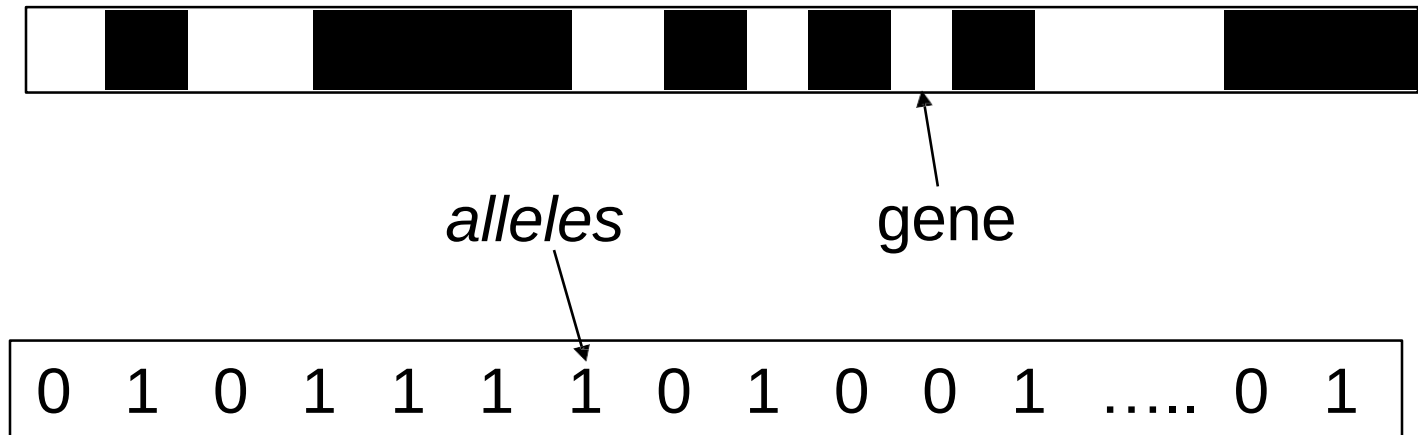


Generation n

Generation $n+1$

GA Terminology: Chromosomes

Chromosome (string)



- Each chromosome represents a solution, often using strings of 0's and 1's. Each bit typically corresponds to a gene. This is called binary encoding.
- The values for a given gene are the alleles.

A chromosome in isolation is meaningless - need decoding of the chromosome into phenotypic values.

GA Terminology: Encoding–Decoding

genotype

coded domain

phenotype

decision domain

Biology

UGCAACCGU
("DNA" blocks)

expression

sequencing

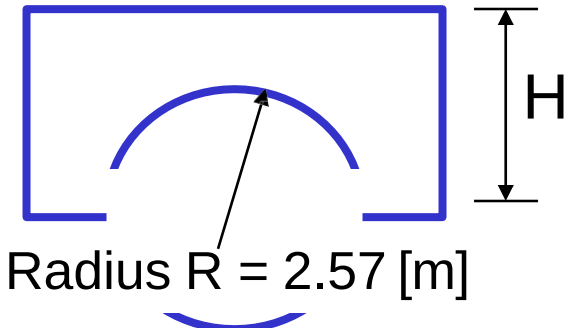
“blue eye”

Design

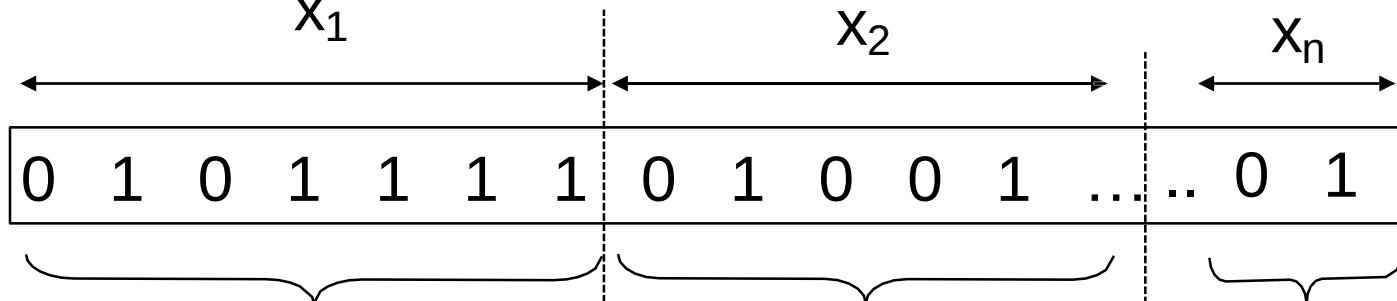
10010011110
(chromosome)

decoding

encoding



Radius $R' = 2.57$ [m]

$$x_1$$
$$X_2$$
 x_n 

Radius (genotype)

Height

Material

Genetic Algorithms(GA) OVERVIEW

The Metaphor(cont.)

The computer model introduces simplifications (*relative to the real biological mechanisms*),

.. BUT ..

Surprisingly complex and interesting structures have emerged out of evolutionary algorithms.

Example..

the MAXONE problem

Suppose we want to maximize the number of ones in a string of l binary digits

Is it a trivial problem?

It may seem so because we know the answer in advance.

However, we can think of it as maximizing the number of correct answers, each encoded by 1, to l yes/no difficult questions.

Example..

the MAXONE problem

- An individual is encoded (naturally) as a string of l binary digits.
- The fitness f of a candidate solution to the MAXONE problem is the number of ones in its genetic code.
- We start with a population of n random strings.
Suppose that $l = 10$ and $n = 6$.

Example..

the MAXONE problem.. Initialization

We toss a fair coin 60 times and get the following initial population:

$$s_1 = 1111010101 \quad f(s_1) = 7$$

$$s_2 = 0111000101 \quad f(s_2) = 5$$

$$s_3 = 1110110101 \quad f(s_3) = 7$$

$$s_4 = 0100010011 \quad f(s_4) = 4$$

$$s_5 = 1110111101 \quad f(s_5) = 8$$

$$s_6 = 0100110000 \quad f(s_6) = 3$$

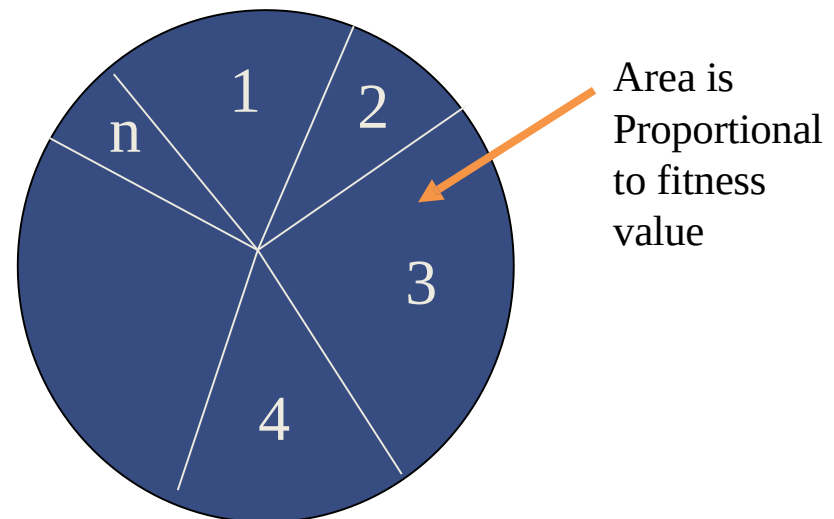
Example..

the MAXONE problem.. Selection

Next we apply fitness proportionate selection with the roulette wheel method:

Individual i will have a probability to be chosen $\frac{f(i)}{\sum_i f(i)}$

We repeat the extraction as many times as the number of individuals we need to have the same parent population size (6 in our case).



Example..

the MAXONE problem.. Selection

Suppose that, after performing selection, we get the following population:

$$s_1' = 1111010101 \quad (s_1)$$

$$s_2' = 1110110101 \quad (s_3)$$

$$s_3' = 1110111101 \quad (s_5)$$

$$s_4' = 0111000101 \quad (s_2)$$

$$s_5' = 0100010011 \quad (s_4)$$

$$s_6' = 1110111101 \quad (s_5)$$

Example..

the MAXONE problem.. Crossover(Recombination)

Next we mate strings for crossover. For each couple we decide according to crossover probability (for instance 0.6) whether to actually perform crossover or not.

Suppose that we decide to actually perform crossover only for couples (s_1, s_2) and (s_5, s_6) . For each couple, we randomly extract a crossover point, for instance 2 for the first and 5 for the second.

Example..

the MAXONE problem.. Crossover (Recombination)

Before crossover:

$$s_1^{\text{'}} = 11\mathbf{11010101}$$

$$s_2^{\text{'}} = 11\mathbf{10110101}$$

$$s_5^{\text{'}} = 01000\mathbf{10011}$$

$$s_6^{\text{'}} = 11101\mathbf{11101}$$

After crossover:

$$s_1^{\text{'}} = 11\mathbf{10110101}$$

$$s_2^{\text{'}} = 11\mathbf{11010101}$$

$$s_5^{\text{'}} = 01000\mathbf{11101}$$

$$s_6^{\text{'}} = 11101\mathbf{10011}$$

Example..

the MAXONE problem.. Mutation

The final step is to apply random mutation: for each bit that we are to copy to the new population we allow a small probability of error (for instance 0.1)

Before applying mutation:

$$s_1'' = 11101\textcolor{violet}{1}0101$$

$$s_2'' = 1111\textcolor{violet}{0}1010\textcolor{violet}{1}$$

$$s_3'' = 11101\textcolor{violet}{1}11\textcolor{violet}{0}1$$

$$s_4'' = 0111000101$$

$$s_5'' = 0100011101$$

$$s_6'' = 11101100\textcolor{violet}{1}1$$

Example..

the MAXONE problem.. Mutation

After applying mutation:

$$s_1''' = 1110100101 \quad f(s_1''') = 6$$

$$s_2''' = 1111110100 \quad f(s_2''') = 7$$

$$s_3''' = 1110101111 \quad f(s_3''') = 8$$

$$s_4''' = 0111000101 \quad f(s_4''') = 5$$

$$s_5''' = 0100011101 \quad f(s_5''') = 5$$

$$s_6''' = 1110110001 \quad f(s_6''') = 6$$

Example..

the MAXONE problem..

In one generation, the total population fitness changed from 34 to 37, thus improved by ~9%.

At this point, we go through the same process all over again, until a stopping criterion is met.

Components of GA..

the Overall Components..

A problem definition as input, and:

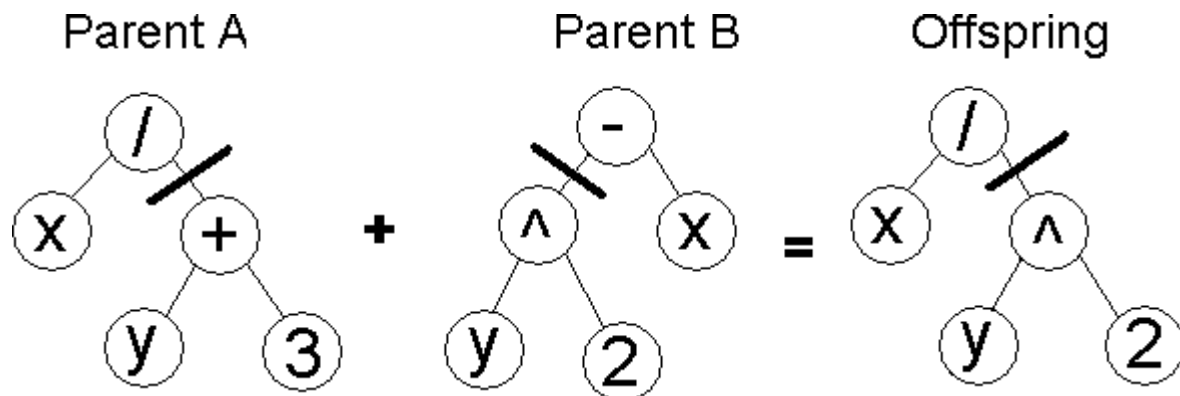
- Encoding principles *(gene, chromosome)*
- Initialization procedure *(creation)*
- Selection of parents *(reproduction)*
- Genetic operators *(mutation, recombination)*
- Evaluation function *(environment)*
- Termination condition

Components of GA..

Representation (Encoding)

Possible individual's encodings:

Bit strings	(0101 .. 1100)
Real numbers	(43.2 -33.1 .. 0.0 89.2)
Permutations of element	(E11 E3 E7 .. E1 E15)
Lists of rules	(R1 R2 R3 .. R22 R23)
Program elements	(genetic programming)
... any data structure ...	



Components of GA..

Representation (Encoding)

When choosing an encoding method rely on the following key ideas:

1. Use a data structure as close as possible to the natural representation.
2. Write appropriate genetic operators as needed.
3. If possible, ensure that all genotypes correspond to feasible solutions.
4. If possible, ensure that genetic operators preserve feasibility.

Components of GA..

Initialization

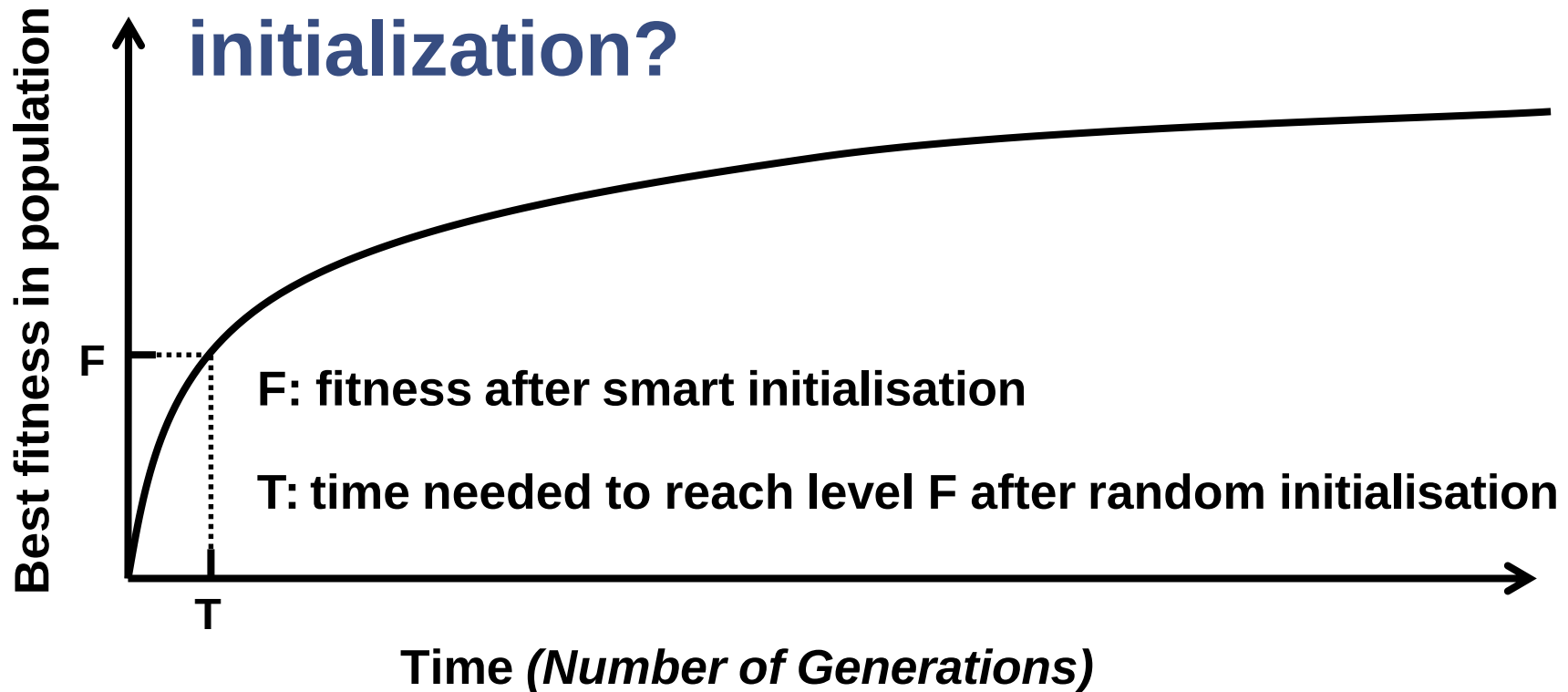
Start with a population of randomly generated individuals, or use:

- A previously saved population.**
- A set of solutions provided by a human expert.**
- A set of solutions provided by another heuristic algorithm.**

Components of GA..

Initialization

Is it worth spending effort on smart initialization?



Answer:

- it depends. Possibly, if good solutions/methods exist ..

Components of GA..

Selection

Purpose: to focus the search in promising regions of the space ..

Inspiration: Darwin's "survival of the fittest" ..

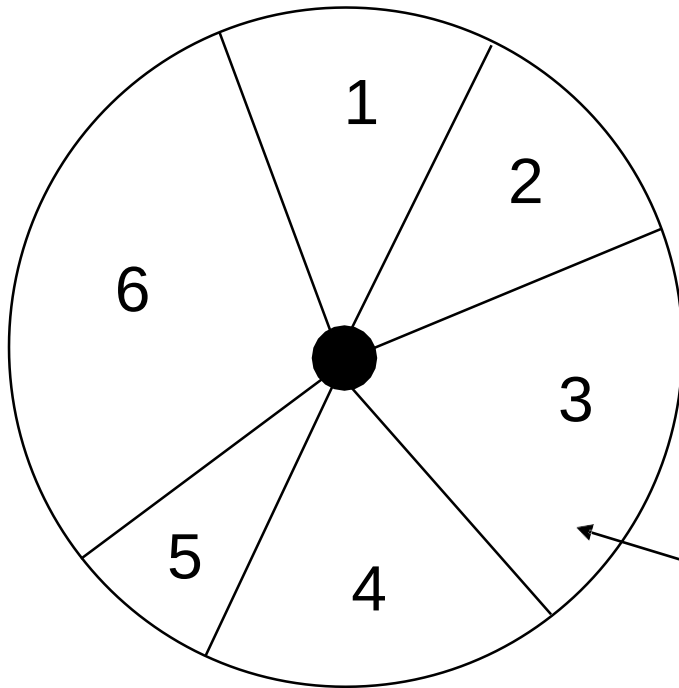
Trade-off: between *exploration* and *exploitation* of the search space ..

Next we shall discuss possible selection methods.

Components of GA..

Selection.. *Roulette Wheel Selection*

(1) Roulette Wheel Selection



Probabilistically select individuals based on some measure of their performance.

Sum Sum of individual's selection probabilities

3rd individual in current population mapped to an interval in $[0, Sum]$

Selection: generate random number in $[0, Sum]$

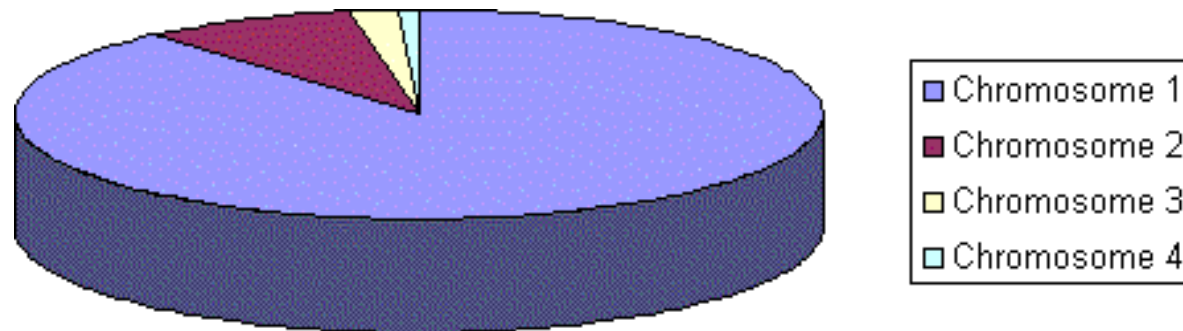
Repeat process until desired # of individuals selected.

Components of GA..

Selection.. *Roulette Wheel Selection*

This can be simulated by following algorithm.

1. **[Sum]** Calculate sum of all chromosomes' fitnesses in the population - sum S .
2. **For** each individual to be selected, do the following:
 - i. **[Select]** Generate a random number r from the interval $[0, S]$.
 - ii. **[Loop]** Go through the population and sum the fitnesses one by one starting from 0 in sum s . When the sum s is greater than r , stop and return the chromosome where you are.



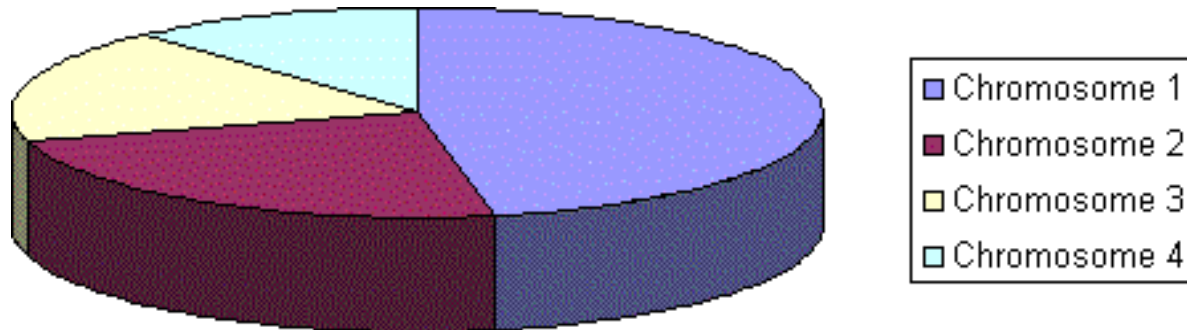
Components of GA..

Selection..by Ranking

(2) Selection according to RANKING

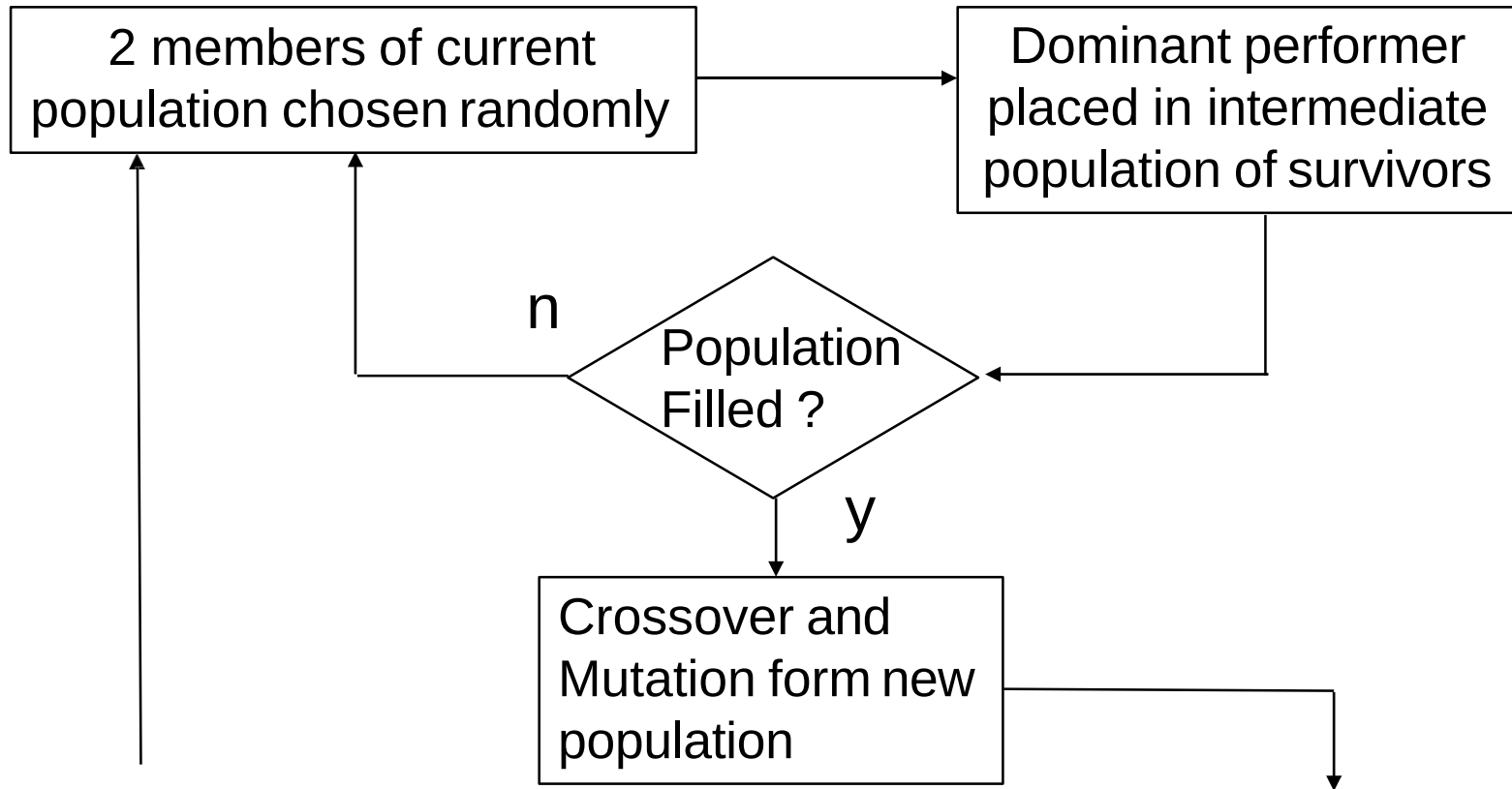
Example: Let $D = \sum_{j \in P} 1/j$
select the k^{th} most fit member of a population
to be a parent with probability $P_k = \left(\frac{1}{k}\right) D^{-1}$

⇒ Better ranking has a higher probability of being chosen,
e.g. $1^{\text{st}} \propto 1$, $2^{\text{nd}} \propto 1/2$, $3^{\text{rd}} \propto 1/3 \dots$



Components of GA..

Selection.. *Tournament Selection*



Old Population	Fitness
101010110111	8
100100001100	4
001000111110	6

Survivors	Fitness
101010110111	8
001000111110	6
101010110111	8

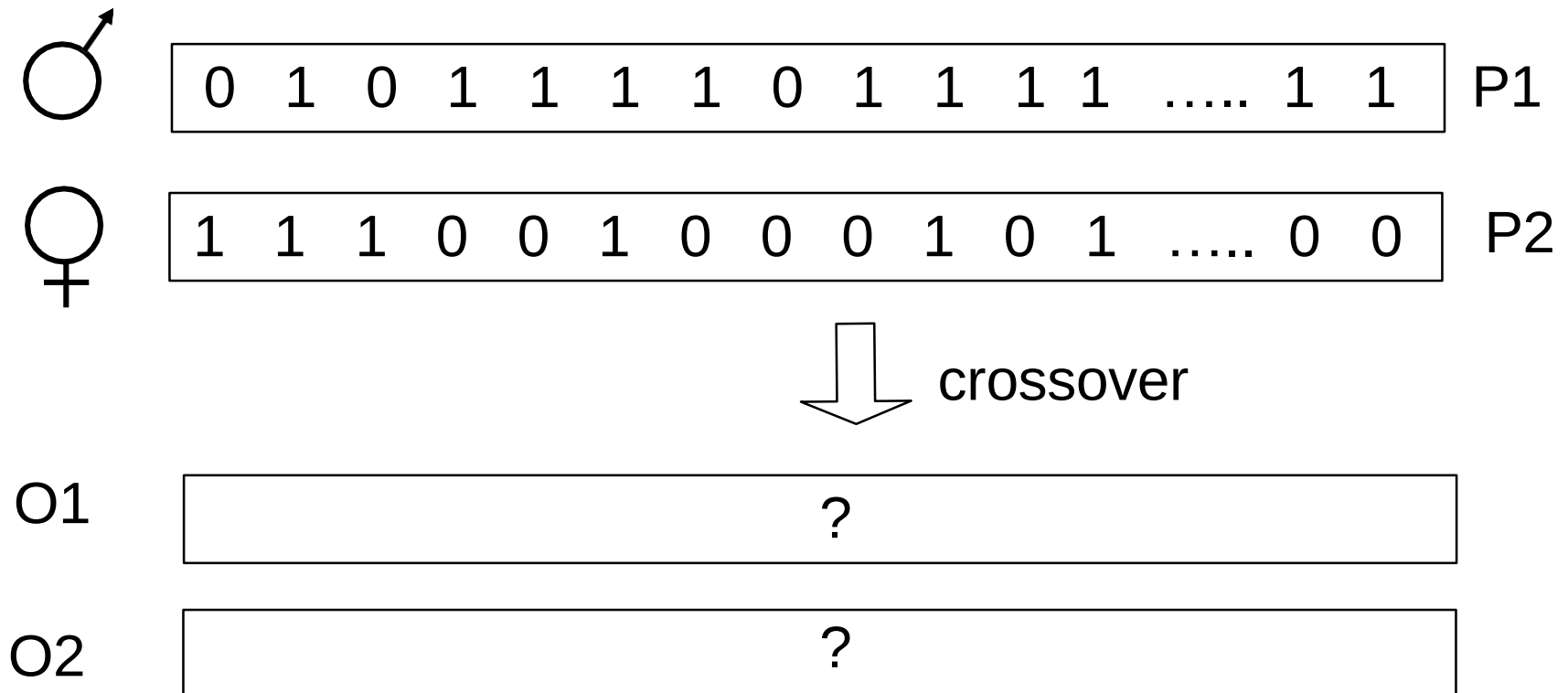
Components of GA..

Recombination(Crossover)

- Enables the evolutionary process to move toward promising regions of the search space.
- Matches good parents' sub-solutions to construct better offspring.

Components of GA..

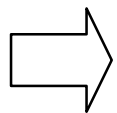
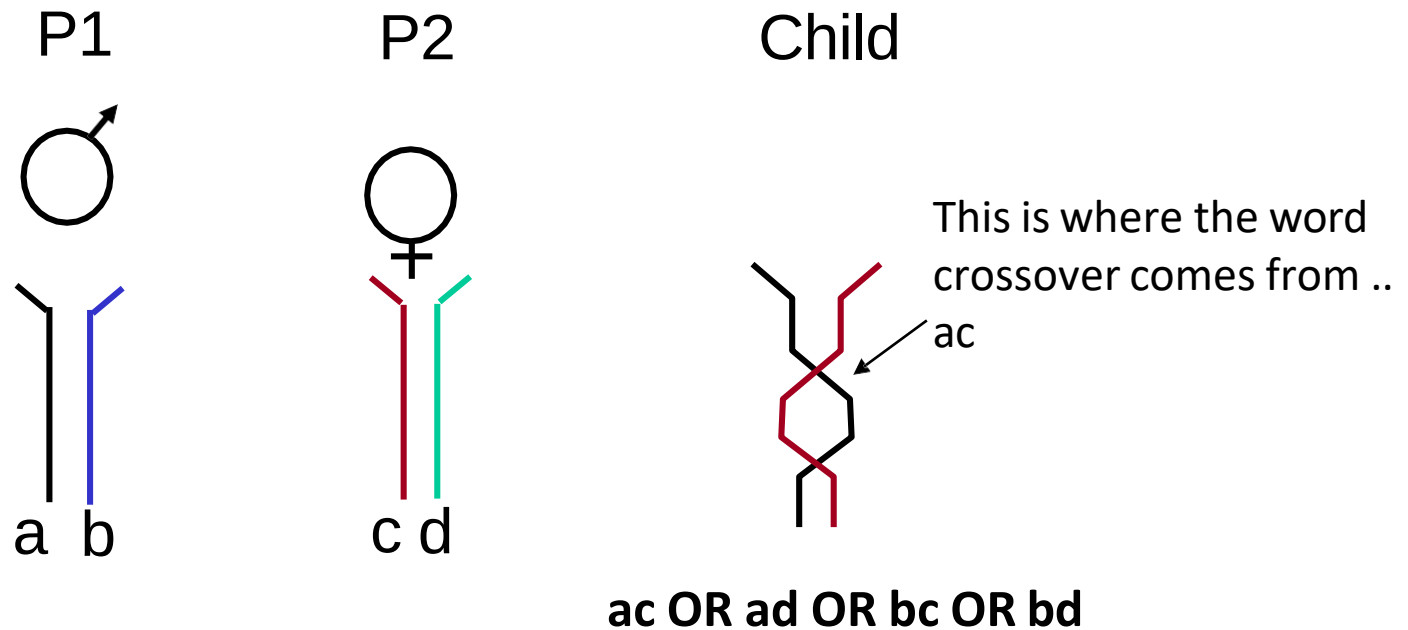
Recombination(Crossover)..*in Biology*



Question: How can we operate on parents P1 and P2 to create offspring O1 and O2 (same length, only 1's and 0's)?

Components of GA..

Recombination(Crossover)..in Biology



Crossover produces either of these results for each chromosome.

Components of GA..

Recombination (Crossover) ..

Crossover (mating) is taking 2 solutions, and creating 1 or 2 more

Classical: single point crossover

P1	0	1	1	0	1
P2	1	0	0	1	1

The parents

crossover point

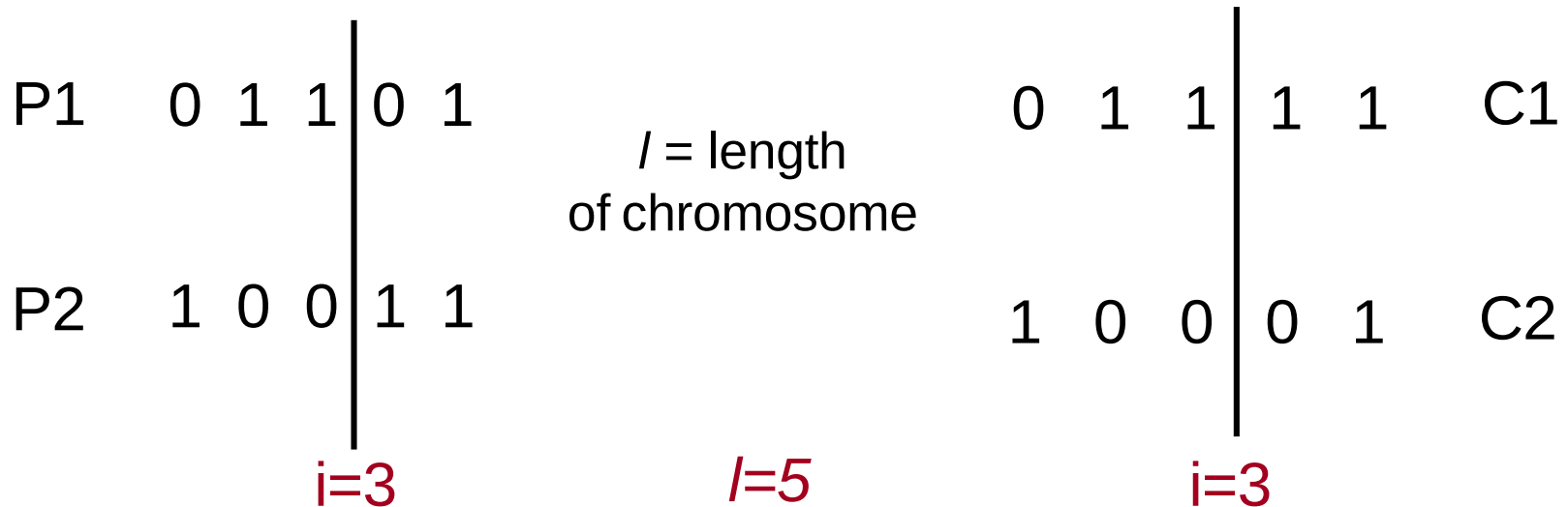
0	1	1	1	1	O1
1	0	0	0	1	O2

The children
("offspring")

Components of GA..

Recombination (Crossover) ..

More on 1-point crossover ..



A crossover bit “ i ” is chosen (deliberately or randomly), splitting the chromosomes in half.

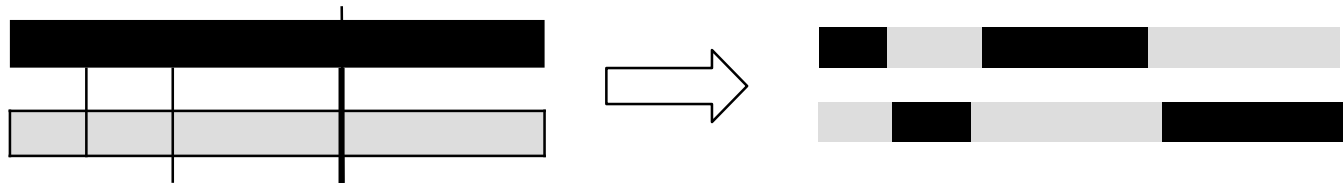
Child C1 is the 1st half of P1 and the 2nd half of P2 ..

Child C2 is the 1st half of P2 and the 2nd half of P1 ..

Components of GA..

Recombination(Crossover)..

- One can also do a 2-point crossover or a multi-point crossover.



- The essential aspect is to create at least one child (solution/design) from two (or more) parent (solutions/designs).
- There are many solutions to do this.

Components of GA..

Mutation

Purpose: to simulate the effect of errors that happen with low probability during duplication.

Results:

- Movement in the search space.
- Restoration of lost information to the population.

Components of GA..

Elitism

Purpose: When creating a new population by crossover and mutation, we have a big chance, that we will lose the best chromosome. Elitism is a method which first copies the best chromosome (*or a few best chromosomes*) to the new population. The rest is done in classical way.

Results:

Elitism can very rapidly increase performance of GA, because it prevents losing the best found solution.

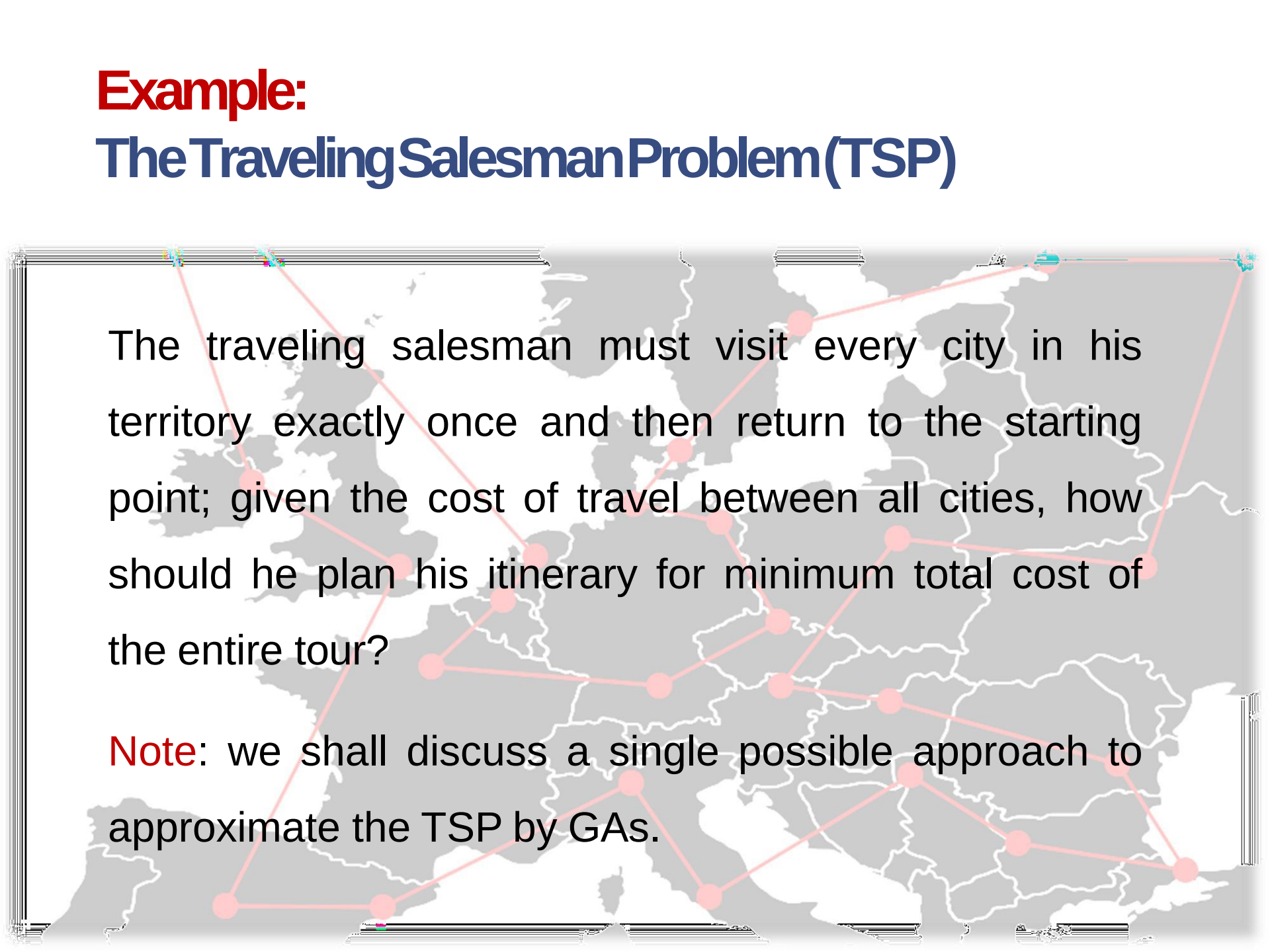
Components of GA..

Evaluation (Fitness Function)

- **Solution is only as good as the evaluation function; choosing a good one is often the hardest part.**
- **Similar-encoded solutions should have a similar fitness.**

Example:

The Traveling Salesman Problem (TSP)

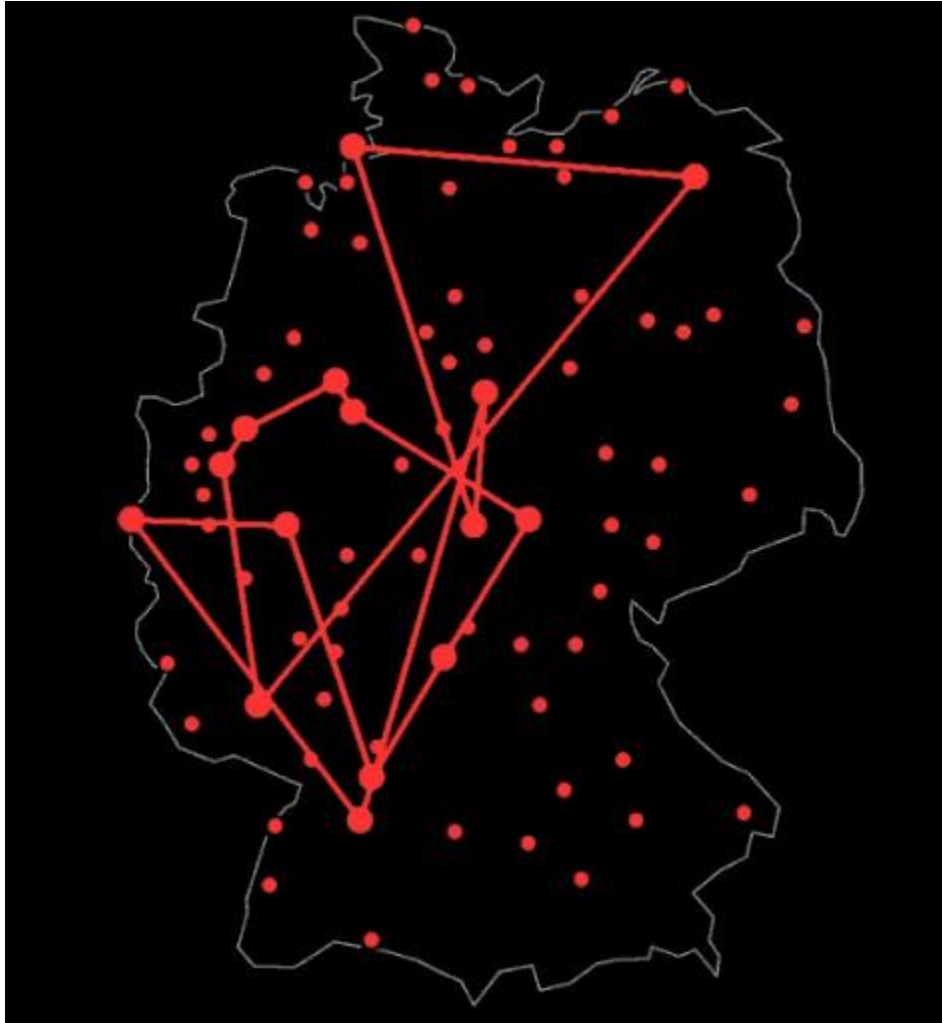


The traveling salesman must visit every city in his territory exactly once and then return to the starting point; given the cost of travel between all cities, how should he plan his itinerary for minimum total cost of the entire tour?

Note: we shall discuss a single possible approach to approximate the TSP by GAs.

Example:

The Traveling Salesman Problem (TSP)



Example:

The Traveling Salesman Problem (TSP)

(Representation, Evaluation, Initialization & Selection)

A vector $v = (i_1 i_2 \dots i_n)$ represents a tour (v is a permutation of $\{1, 2, \dots, n\}$).

Fitness f of a solution is the inverse cost of the corresponding tour.

Initialization: use either some heuristics, or a random sample of permutations of $\{1, 2, \dots, n\}$.

We shall use the fitness proportionate selection.

Example:

The Traveling Salesman Problem (TSP) (Crossover)

Build an offspring by choosing a sub-sequence of a tour from one parent and preserving the relative order of cities from the other parent and feasibility.

Example:

$p_1 = (1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9)$ and $p_2 = (4\ 5\ 2\ 1\ 8\ 7\ 6\ 9\ 3)$

First, the segments between cut points are copied into offspring:

$o_1 = (x\ x\ x\ 4\ 5\ 6\ 7\ x\ x)$ and $o_2 = (x\ x\ x\ 1\ 8\ 7\ 6\ x\ x)$

Example:

The Traveling Salesman Problem (TSP) (Crossover)

Next, starting from the second cut point of one parent, the cities from the other parent are copied in the same order.

The sequence of the cities in the second parent is:

9 – 3 – 4 – 5 – 2 – 1 – 8 – 7 – 6

After removal of cities from the first offspring we get:

9 – 3 – 2 – 1 – 8

This sequence is placed in the first offspring:

$o_1 = (2\ 1\ 8\ 4\ 5\ 6\ 7\ 9\ 3)$, and similarly in the second:

$o_2 = (3\ 4\ 5\ 1\ 8\ 7\ 6\ 9\ 2)$

Example:

The Traveling Salesman Problem (TSP)

(Mutation: *Inversion*)

The sub-string between two randomly selected points in the path is reversed.

Example:

(1 2 3 4 5 6 7 8 9) is changed into (1 2 7 6 5 4 3 8 9)

Such simple inversion guarantees that the resulting offspring is a legal tour.

Thanks! ... *Questions?*